

Privatezza e Protezione Dati

simi

September 2024

Contents

1	Introduzione	4
2	Privacy and Data Publication	6
2.1	Macrodati	7
2.1.1	Protezione identità	8
2.1.2	Informazioni sensibili	8
2.1.3	Vedere correlazioni	8
2.1.4	Restringere l'accesso	8
2.2	Problema Anonimità	9
2.2.1	Misura di rischio	9
2.3	k-anonymity	10
2.3.1	Domain generalization hierarchy	11
2.3.2	Value generalization hierarchy	11
2.3.3	Generalizzazione minima	12
2.3.4	Classificazione di tecniche k-anonymity	13
2.3.5	Algoritmi per calcolare la tabella k-anonymous	14
2.3.6	Algoritmi Euristici	16
2.3.7	k-anonymity revisited	17
2.4	Divulgazione degli attributi	18
2.4.1	Problema di omogeneità	18
2.4.2	Background knowledge	18
2.4.3	l-diversity	19
2.4.4	Skewness attack	19
2.4.5	Similarity attack	19
2.4.6	Group closeness	19
2.4.7	External Knowledge modeling	19
2.4.8	Rilasci multipli	20
2.4.9	m-invariance	20
2.4.10	Extended scenarios	20
2.5	k-anonymity in vari scenari	21
2.5.1	Social Networks	21
2.5.2	Data mining	22
2.5.3	Location base service	23
2.6	Re-identificazione	23
2.7	Privacy semantica	24
2.7.1	Differenze con k-anonymity	24

2.8	Sensitive Value Distributions	25
2.9	Dati genomici	25
2.10	Sensitive inference from data mining	25
2.11	Social Media	25
2.11.1	Cambridge analitica	26
3	Protezione Macrodati e Microdati	27
3.1	Statistical data dissemination	28
3.2	Statistical DBMS	29
3.3	Macrodati	29
3.3.1	Campionamento	30
3.3.2	Regole	30
3.3.3	Protezione celle	31
3.3.4	Protection of tables of magnited data	33
3.4	Microdati	35
3.4.1	Sampling	36
3.4.2	Soppressione	36
3.4.3	Global recoding	36
3.4.4	top-coding o bottom-coding	36
3.4.5	Generalizzazione	36
3.4.6	Random noise	37
3.4.7	Swapping	37
3.4.8	Micro-aggregation	38
3.4.9	Tecniche sintetiche	39
4	Privacy e protezione dei dati in scenari emergenti	40
4.1	Cloud Computing	41
4.2	Protezione dati in contesto Cloud	42
4.2.1	Security properties	43
4.2.2	Access Requirements	43
4.2.3	Architettura	44
4.3	Digital Data Market	45
4.4	Problemi da guardare	45
5	Preferenze di privacy degli utenti	46
5.1	User empowerment	47
5.2	User privacy preferences: Some approaches	50
5.2.1	Cost-sensitive trust negotiation	50
5.2.2	Point-based trust management model	51
5.2.3	Logic-based trust management	52
5.2.4	Privacy Preferences in Credential-based Interactions	54
5.2.5	Disclure	55
5.2.6	Sensibilità portfolio	56
5.2.7	Server-side issues	59

6	Protezione dei dati in outsourcing	60
6.1	Encryption	61
6.1.1	Keyword-based searches	61
6.1.2	Homomorphic encryption	61
6.1.3	Onion encryption	62
6.1.4	Indexes	62
6.1.5	Query execution	65
6.1.6	Hash based index	65
6.1.7	Order preserving encryption	67
6.2	Inference exposure	67
6.3	Bloom Filter	69
6.4	Data Integrity	70
6.5	Selective Encryption	70
6.5.1	Attribute base encryption	71
6.5.2	Politica di autorizzazione	72
6.6	Collusion Attack	75
6.6.1	Classification of users	75
6.7	Mix & Slice	77
6.8	Write Authorizations	79
6.9	Fragmentation and Encryption	79
6.9.1	Coppia di server non comunicanti	80
6.9.2	Identifying the optimal decomposition	82
6.9.3	Multiple non-linkable fragments	82
6.10	Fragmentation	84
6.10.1	Query evaluation	85
6.10.2	Minimal fragmentation	87

Chapter 1

Introduzione

Quello che vuole fare il corso è dare una sensibilizzazione sul problema di protezione dati, inizialmente ci focalizzeremo sui problemi di privacy, non è semplice fare questo, in quanto non basta solamente proteggere i dati sensibili, ma anche i dati di contesto. Il corso non ha l'obiettivo di dare una carrellata di concetti ma è importante fornire degli strumenti per poter trovare delle soluzioni.

Cominciamo con il voler proteggere la confidenzialità dei dati, il problema è il rilascio di informazioni relative ad entità (persone e aziende). La realtà è che volenti o nolenti ci sono un sacco di dati in giro, ci sono tanti dati che generiamo anche noi (dai social network).

Voler proteggere i dati non vuole dire essere contro la condivisione, perchè condividere i dati serve (ricerche mediche su dati dei pazienti). Il problema è che devo garantire che i dati siano protetti, per questo sta emergendo la possibilità di usare servizi di terze parti. Proteggere la privacy nel rilascio dei dati in modo pubblico o semi-pubblico e proteggere la privacy dei dati (anche di utenti e accessi ai dati) quando i dati sono memorizzati su terze parti.

Chapter 2

Privacy and Data Publication

Una prima idea potrebbe essere di rilasciare i dati aggregati (la media), qui ci sono due filoni:

- statistical DBMS: hai i tuoi dati e il DBMS che si interfaccia ai dati per l'accesso e gli utenti che si interfacciano al DBMS, il quale non risponde a query che richiedono informazioni puntuali o dati sensibili, ma risponde solo a query che chiedono dati aggregati. Qui ho due problemi, che vi sono sempre in qualsiasi controllo di sicurezza dinamico:
 - La finestra di osservazione: se devo ricordarmi l'accumulo di conoscenza che hai devo ricordarmi quello che ti dico (perchè devo non poterti fare aggregazione delle cose che ti dico)
 - Collisione: se devo controllare l'accumulo di conoscenza che hai controllo te singolo individuo o chi è fuori per me è un mondo unico?
- statistical Data: precomputare le informazioni statistiche e gli utenti accedono a quelle informazioni. Calcolo le statistiche che voglio rilasciare quindi le query non colpiscono direttamente il DBMS.

I microdati sono dati grezzi, a volte potremmo rilasciare row data (la tabella precisa ma senza nomi dei pazienti).

2.1 Macrodati

- Tabelle di conteggio o frequenza, quando ciascun record mi riporta il conteggio di quanti respondents ho dietro ai dati (quanti utenti)
- Tabelle di grandezza: mi dicono un aggregato sulla quantità di interesse della mia analisi

Esempio tabella di conteggio, ogni riga mi dice quante persone per dipartimento ho in quella fascia:

Dept	Income						Total
	[0-21)	[21-23)	[23-25)	[25-27)	[27-29)	29+	
Dept ₁	2	4	18	20	7	1	52
Dept ₂	-	-	7	9	-	-	16
Dept ₃	-	6	30	15	4	-	55
Dept ₄	-	-	2	-	-	-	2

Nelle tabelle di grandezza abbiamo delle computazioni, nell'esempio vediamo i maschi e le femmine in media quanti giorni passano in ospedale per ogni malattia:

	Hypertension	Obesity	Chest Pain	Short Breath	Tot
M	2	8.5	23.5	3	37.0
F	3	30.5	0	5	38.5
Tot	5	39.0	23.5	8	75.5

Una tabella di microdati consiste nel rilasciare tutti i dati:

N	Child	County	Educ. HH	Salary HH	Race HH
1	John	Alfa	very high	201	black
2	Jim	Alfa	high	103	white
3	Sue	Alfa	high	77	black
4	Pete	Alfa	high	61	white
5	Ramesh	Alfa	medium	72	white
6	Dante	Alfa	low	103	white
7	Virgil	Alfa	low	91	black
8	Wanda	Alfa	low	84	white
9	Stan	Alfa	low	75	white
10	Irmi	Alfa	low	62	black
11	Renee	Alfa	low	58	white
12	Virginia	Alfa	low	56	black
13	Mary	Alfa	low	54	black
14	Kim	Alfa	low	52	white
15	Tom	Alfa	low	55	black
16	Ken	Alfa	low	48	white
17	Mike	Alfa	low	48	white
18	Joe	Alfa	low	41	black
19	Jeff	Alfa	low	44	black
20	Nancy	Alfa	low	37	white

Cosa devo proteggere se rilascio tabelle di questo tipo?

- Identità di chi sta dietro ai dati, a volte solo anche essere nel dataset è sensibile
- Informazioni sensibili, sono informazioni che dovrebbero essere mantenute private
- Capire che ci sono certe correlazioni sui dati, posso avere bisogno di bloccare queste cose

2.1.1 Protezione identità

Per i macrodati potrebbe non essere un problema che un individuo fa parte dei macrodati finché l'informazione che rilascia il macrodato non può essere inferito all'utente. Per i microdati il fatto di sapere che sei nel dataset che viene rilasciato è più problematico perché è più facile che esponga l'informazione sensibile.

2.1.2 Informazioni sensibili

Mi pongo questo problema anche quando chi riceve i dati sa inferire l'informazione con una confidenza abbastanza preciso.

2.1.3 Vedere correlazioni

Il problema delle correlazioni è che quando le scopri sono canali di inferenza che puoi sfruttare.

2.1.4 Restringere l'accesso

Per proteggere le informazioni sulla privacy sicuramente è rimuovere le informazioni identificanti. Quando mi pongo il problema di privacy ho un problema di andare ad operare tutti i dati in modo che non comunichino informazioni sensibili ma posso usare anche un accesso ristretto ai dati, non do tutta l'informazione ma restringo alcuni dati. Posso mettere delle restrizioni sull'accesso, rilasciamo i dati per un certo scopo e chi li riceve li deve usare per quello scopo.

Per tabelle di conteggio e frequenza non faccio le statistiche su tutti i dati ma le faccio su un campione, poi posso avere delle regole di soglia che mi restringono i dettagli che posso rilasciare.

Per i microdati ho delle tecniche per sanificare i dati (per poterli pubblicare):

- non-perturbative: tecniche che non sporcano i dati, ma fanno perdere dettagli
- perturbative: sporchiamo i dati. Dobbiamo stare attenti alla propagazione del rumore.

Un'altra tecnica è generare dati sintetici che rispecchiano un profilo di quelli veri e possono servire per fare testing dell'applicazione.

2.2 Problema Anonimità

Vedremo le tecniche classiche di protezioni che sfruttano tecniche moderne. De-identificare i dati non basta per garantirne la protezione, il problema sono i linking attach che permettono di re-identificare i dati (dato che esistono un sacco di dati su di noi). Anche quando non re-identificano la persona correttamente rimane un problema perchè riducono l'incertezza.

Il problema è che quando proteggero i dati il fatto che ci sia informazione che identifica le persone è noto, sono informazioni identificanti. Le informazioni confidenziali sono quelli sensibili che dobbiamo proteggere. Abbiamo informazione non confidenziali di cui non mi interessa niente, ad oggi è quasi vuoto questo insieme, perchè ci sono dati che possono essere combinati per risalire all'individuo. Esistono anche informazioni quasi identificanti che riducono l'incertezza sulle persone che stanno dietro ai dati (zip, data di nascita..). Poi abbiamo gli attributi confidenziali, ovvero quelli che contengono informazioni sensibili e gli identifiers che sono attributi che individuano un singolo individuo.

Abbiamo visto uno studio in cui dice che la combinazione dettagliata di proprietà semi-identificanti ci permette di dettagliare molte persone, più perdiamo dettaglio più andiamo giù, questo problema viene chiamato re-identification.

Chiaramente non tutti siamo esposti allo stesso modo, sarà più esposto chi ha dei valori peculiari rispetto ad una situazione (un anziano ricoverato in un ospedale pediatrico). I rischi che hai è se chi osserva i dati ha a disposizione altri strumenti esterni che ci riguardano.

Hai più rischio di esposizione più attributi hai in comune, più sorgenti esterne ci sono e più di dati sono dettagliati. Ti può aiutare che i dati rilasciati sono un sottoinsieme delle informazioni che ci sono, i dati potrebbero non essere aggiornati (ci sono dati soggetti ad embargo), posso aver anche errori di formato di rappresentazione.

2.2.1 Misura di rischio

Quando devo proteggere i dati devo proteggere tutti, perchè tendenzialmente chi cerca i dati non cerca qualcuno in particolare. Se voglio parlare di rischio di rilascio improprio dell'informazione è:

- se quello che stai cercando è sia nei miei dati che in quelli fuori

- se i dati sono correlabili, sono rappresentati in un formato che possono essere correlati
- la probabilità che tu stia cercando sia abbastanza peculiare per permettere di fare la correlazione

Abbiamo detto che più informazioni abbiamo e più queste sono dettagliate più la gente sarà esposta.

2.3 k-anonymity

Il grado di protezione che mi dà dove lo fisso? sono in grado di misurarlo?

È come misura la protezione che sto dando ai miei utenti. Cercano di proteggere identità dei rispondenti rilasciando informazione vera, stiamo dicendo che usiamo tecniche che non sporcano i dati. Il goal è che non ci siano situazioni in cui ho attributi quasi identificanti che possono essere sfruttati per attacchi di correlazione che disfa l'anonimizzazione che ho fatto.

Quello che mi piacerebbe è che quando guardi i dati non riesci a re-identificare o ridurre l'incertezza rispetto a chi quei dati si riferiscono. La domanda è come calcolo questa threshold, come calcoliamo il grado di privacy che ti do (la cardinalità delle persone a cui può corrispondere un dato). Specifichiamo che non esiste sicurezza in senso assoluto così come non esiste privacy in senso assoluto. L'assunzione è che quello che l'osservatore può sfruttare gli attributi in comune tra i dati che rilascio e quelli che può trovare fuori in altre sorgenti. Mi sto proteggendo così da attacchi che sfruttano informazioni quasi identificanti. Questo è quello che vorrei fare, come lo traduco nel controllo da fare? capito il problema ci chiediamo come lo traduciamo in una proprietà sul rilascio (se quello da cui ci vogliamo proteggere è una correlazione tra i nostri dati e quelli fuori, se per ogni occorrenza abbiamo tante tuple con lo stesso valore questo tipo di correlazione non può essere fatta) e come facciamo a tradurre quella proprietà. Ogni rilascio di dati deve essere fatto in modo che per ciascuna occorrenza di valori devono essere matchati almeno k esponenti, si traduce che ciascuna occorrenza deve apparire k volte.

Il problema è che il contenuto dei dati non dipende da me, posso applicare una protezione in modo che sia così. Ci sono due (o due famiglie) che non perturbano i dati:

- Generalizzazione: se invece di pubblicare la data di nascita pubblichiamo il mese e l'anno (intervallo). Se facciamo perdere di dettaglio anche gente che è diversa diventa uguale. Questo lo posso fare in vari modi, come se avessi una gerarchia di generalizzazioni (giorno, mese, anno; mese, anno; anno)
- Soppressione: proteggere informazioni sensibili rimuovendole. Non mi perturba effettivamente i dati, ma non mi dà il dato nella sua interezza.

Usiamo la generalizzazione e chiediamo aiuto alla soppressione quando abbiamo degli outlier troppo distanti tra di loro quando dovrei aumentare troppo l'intervallo di generalizzazione.

2.3.1 Domain generalization hierarchy

Per ciascun attributo che abbiamo abbiamo una gerarchia di domini (5 digit il meno generalizzato e si va a scalare 4, 3, 2...). Ad esempio per le razze posso avere le razze e a livello sopra siamo tutte persone.

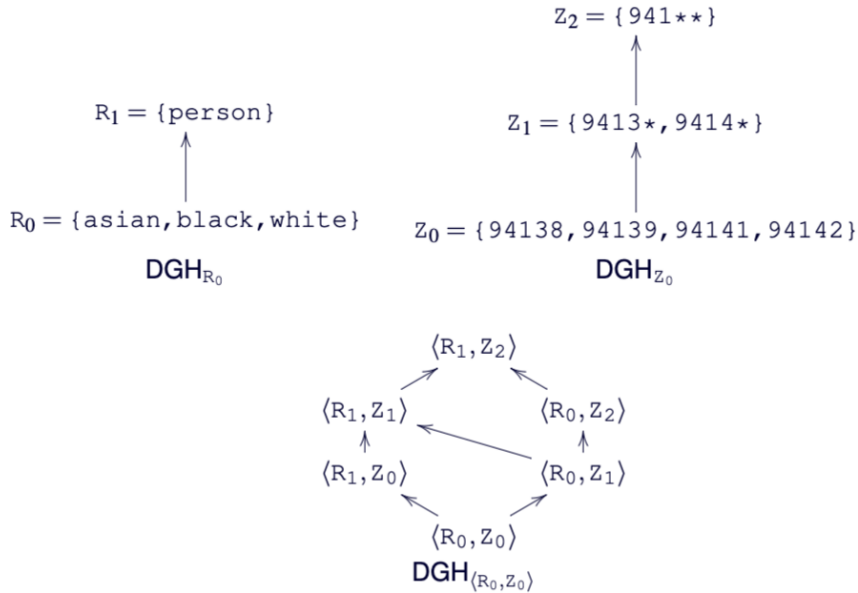
Quindi ho una gerarchia di generalizzazione per ogni dominio: $DGH_D = (Dom, \leq D)$, possiamo dire che per ogni dominio de Di sta sotto Dj e Di sta sotto Dj allora o Dj sta sotto Dz o Dz sta sotto Dj:

$$\forall D_i, D_j, D_z \in Dom$$

$$D_i \leq D_j, D_i \leq D_z \rightarrow D_j \leq D_z \vee D_z \leq D_j$$

Sostanzialmente stiamo dicendo che la gerarchia ha una relazione d'ordine totale.

Noi abbiamo tutte le combinazioni possibili dei domini generalizzati, dato che abbiamo due ordinamenti totali abbiamo un reticolo, parto da una tabella di base e le altre sono tabelle ottenute dalla generalizzazione degli altri insiemi, ogni link è un passo di generalizzazione di un insieme:



2.3.2 Value generalization hierarchy

Un'altra cosa è generalizzare i valori. Qui abbiamo un'altra assunzione, se vai a livello di valori la gerarchia che avevi sui domini la trasporti sui valori e quello che hai è un albero. Assumiamo che sia un'albero, significa che ogni dominio è un nodo e c'è un solo padre, le foglie sono i valori più generali.

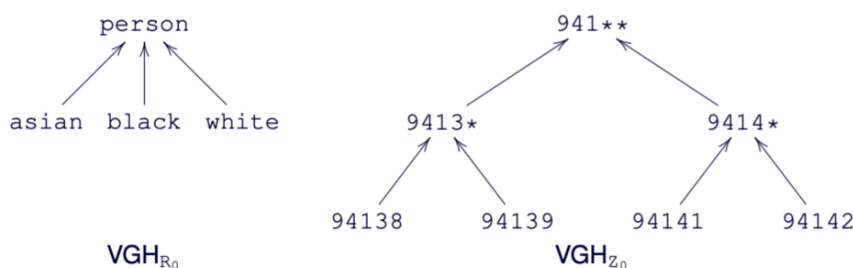
Noi vogliamo che una tabella che rilasciamo è una generalizzazione della tabella principale. Se abbiamo due tabelle T_i e T_j definite sullo stesso set di attributi, possiamo dire che T_j è una generalizzazione di T_i se:

- La cardinalità di $T_j \leq T_i$
- Il dominio $dom(A, T_j)$ di ogni attributo A in T_j è uguale, o una generalizzazione, del dominio $dom(A, T_i)$. Per ogni attributo il dominio è lo stesso o è una generalizzazione.

- Serve stabilire una corrispondenza tra le istanze delle due tabelle (quella normale e quella generalizzata). È possibile definire una funzione iniettiva che associa una tupla di T_j a una di T_i .

Quindi se abbiamo gli attributi con la loro gerarchia e combinando le diverse gerarchie otteniamo un reticolo, se ragioniamo su queste gerarchie ragioneremmo con i passi di generalizzazione (sul singolo attributo è un passo di generalizzazione sul dominio). Inoltre se parlassimo di quanto abbiamo generalizzato diremmo il numero di passi.

Questo ci porta a dire quanto dista una generalizzazione dalla situazione in cui sono partito rispetto a quanti passi di generalizzazione ho fatto, se ho più attributi voglio ragionare di quanti passi ho fatto su n attribuiti. Definiamo il vettore di distanza, che ci permette di misurare quanto abbiamo generalizzato, 2 passi su race e 0 passi su ZIP.



2.3.3 Generalizzazione minima

Bisogna stare attenti a garantire una protezione, ma non eccessiva. Cosa vuol dire per una tabella generalizzata essere una soluzione minima al problema di k -anonimity? abbiamo una tabella T_j che è una generalizzazione della tabella T_i , generalizzo o sopprimo? finchè stai sotto una certa soglia (percentuale di dati) sopprimi, altrimenti generalizzi le tuple. Una generalizzazione è minima se:

- T_j soddisfa la k -anonimity applicando la minima soppressione richiesta, per ogni altra tabella che è una generalizzazione di quella originale con lo stesso tuo vettore di generalizzazione, la cardinalità della tabella non è più grande di quella di T_j . T_j è la tabella che al suo livello di generalizzazione toglie tutte, ma anche solo, le tuple che sono outlier che non permettono di applicare k -anonimity. A qualsiasi livello di generalizzazione di tabelle minime ce n'è una sola.
- Possiamo sopprimere solo fino alla soglia: $|T_i| - |T_j| \leq MaxSup$. Controlla che viene fatta una soppressione minimale.
- Per ogni T_z che è generalizzazione di T_i e che soddisfa le condizioni 1 e 2 (è la generalizzazione che al suo livello di distanza), non è vero che il suo vettore di distanza V_{iz} è dominato dal vettore di distanza V_{ij} . Qui il problema è che voglio generalizzare il meno possibile. Mi piaci se non c'è un'altra generalizzazione più specifica di te.

In questo esempio nessuna delle due soluzioni è migliore, non sono comparabili. La risposta a quale è la migliore potrebbe essere che dipende dal contesto, a cosa ci servono

MaxSup=2

Race:R ₀	ZIP:Z ₀	Race:R ₁	ZIP:Z ₀	Race:R ₀	ZIP:Z ₁
asian	94142			asian	9414*
asian	94141	person	94141	asian	9414*
asian	94139	person	94139	asian	9413*
asian	94139	person	94139	asian	9413*
asian	94139	person	94139	asian	9413*
black	94138			black	9413*
black	94139	person	94139	black	9413*
white	94139	person	94139		
white	94141	person	94141		
PT		GT _[1,0]		GT _[0,1]	

i dati. Un'altra ipotesi potrebbe essere scegliere quella che fa meno passi di generalizzazione, la prima fa 1 passo, la seconda 1. Però è anche vero che ogni passo di generalizzazione non è uguale, perchè un passo su una gerarchia grossa conta poco, su una gerarchia piccola conta tanto, quindi facciamo una distanza relativa dividendo il passo per la altezza della gerarchia, in questo caso il secondo esempio avrebbe generalizzazione 1/2.

Dal punto di vista di lasciare i dati più discriminanti (divisi in gruppi più piccoli) la generalizzazione di destra è preferibile.

In generale effettivamente dipende dal contesto, se devi fare uno studio che comprenda le location è importante che ci siano.

2.3.4 Classificazione di tecniche k-anonymity

Quando generalizzo generalizzo la colonna e quando sopprimo cancello la tupla, in realtà se voglio posso applicare le tecniche di soppressione ad un livello di granularità più fine, l'idea è che se ho già un livello in cui la gente sembra uguale non ha senso generalizzare su di loro, significa lavorare sulla cella.

Generalization	Suppression			
	Tuple	Attribute	Cell	None
Attribute	AG_TS	AG_AS ≡ AG_	AG_CS	AG_ ≡ AG_AS
Cell	CG_TS not applicable	CG_AS not applicable	CG_CS ≡ CG_	CG_ ≡ CG_CS
None	_TS	_AS	_CS	- not interesting

Sopra è il livello di generalizzazione e a lato la soppressione, dentro abbiamo tutte le combinazioni, le coppie sono livelloGeneralizzazione - livelloSoppressione. Le uguaglianze ci sono perchè stiamo dicendo che la soppressione è uguale ad arrivare al livello root dell'attributo (ad esempio stellina nello ZIP o l'intervallo 0-120 nell'età). Il non applicabile è perchè se siamo d'accordo sull'equivalenza, siamo d'accordo che quando generalizzi arrivi ad un punto in cui tutti i valori collasano (sono uguali), quindi se siamo generalizzare a livello di cella sappiamo di conseguenza anche sopprimere a livello di cella.

Race	DOB	Sex	ZIP		Race	DOB	Sex	ZIP
asian	64/04/12	F	94142		asian	64/04	F	941**
asian	64/09/13	F	94141					
asian	64/04/15	F	94139		asian	64/04	F	941**
asian	63/03/13	M	94139		asian	63/03	M	941**
asian	63/03/18	M	94139		asian	63/03	M	941**
black	64/09/27	F	94138		black	64/09	F	941**
black	64/09/27	F	94139		black	64/09	F	941**
white	64/09/27	F	94139		white	64/09	F	941**
white	64/09/27	F	94141		white	64/09	F	941**
PT					AG_TS			

Che faccia hanno le tabelle che posso produrre in questi modi? in questo caso facciamo generalizzazione a livello cella e soppressione tupla, il vettore di distanza sarebbe $[0,1,0,2]$.

2.3.5 Algoritmi per calcolare la tabella k-anonymous

Il problema adesso è calcolare la soluzione, con generalizzazione di attributi e soppressione di tuple è computazionalmente costoso. Gli algoritmi proposti in letteratura hanno un costo esponenziale nel numero di attributi.

Algoritmi per AG_TS e AG_

Un possibile approccio sfrutta la proprietà di monotonicità. Più generalizzo più gente metto insieme, quindi se prendo e generalizzo sempre di più la mia situazione migliorerà sempre, quindi il fatto di avere questa proprietà significa essere monotoni rispetto a quella proprietà. Se non fai problemi di preferenza (lasciare attributo specifico) quello che potresti fare visto che ragioniamo in termini di distanze (dalla root, 0 generalizzazione) potremmo fare la solita ricerca dicotomica, tagliamo a metà ad $h/2$, se in quel punto non trovo la soluzione significa che sotto posso buttare la metà e cerco nella metà sopra e vado avanti così tagliando lo spazio delle soluzioni a metà.

Vediamo un esempio, al primo passo vanno insieme tutte le tuple dominate da un vettore $[0,1]$. I gruppetti ad un certo livello vanno insieme tuple con vettori di distanza dominate dal vettore di distanza che sto cercando a quel passo.

	Race: R_0	ZIP: Z_0		t_1	t_2	$t_3/t_4/t_5$	t_6	t_7	t_8	t_9
t_1	asian	94142	t_1	[0,0]	[0,1]	[0,2]	[1,2]	[1,2]	[1,2]	[1,1]
t_2	asian	94141	t_2	[0,1]	[0,0]	[0,2]	[1,2]	[1,2]	[1,2]	[1,0]
t_3	asian	94139	t_6	[1,2]	[1,2]	[1,1]	[0,0]	[0,1]	[1,1]	[1,2]
t_4	asian	94139	t_7	[1,2]	[1,2]	[1,0]	[0,1]	[0,0]	[1,0]	[1,2]
t_5	asian	94139	t_8	[1,2]	[1,2]	[1,0]	[1,1]	[1,0]	[0,0]	[0,2]
t_6	black	94138	t_9	[1,1]	[1,0]	[1,2]	[1,2]	[1,2]	[0,2]	[0,0]
t_7	black	94139								
t_8	white	94139								
t_9	white	94141								

La gerarchia degli attributi non deve essere per forza un albero, ma se lo è sappiamo per certo che viene rispettata la monotonia, altrimenti no perchè potrebbe essere che

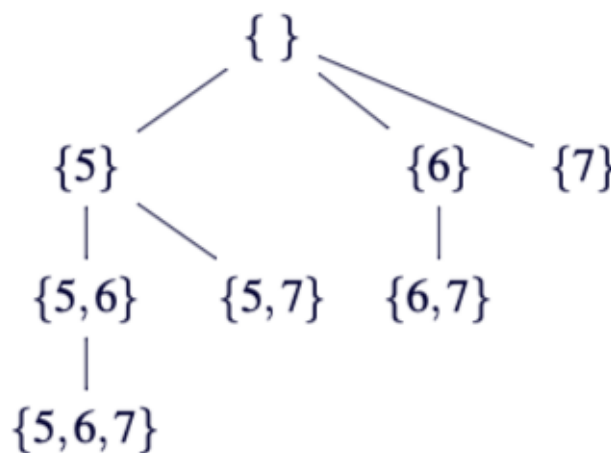
salendo non miglioro la k -anonymity.

k-Optimize algorithm [BA-05]

Assumo che in ciascun dominio gli attributi siano ordinati. Quando va a generalizzare fanno cluster. La rappresentazione è questa, l'ultimo valore del dominio viene omissso, ad esempio:

- Race: {1}, that is: [asian or black or white]
- ZIP: {4, 6}, that is: [94138 or 94139],[94141 or 94142]

Per tagliare l'albero di soluzioni cercano di fare un albero delle soluzioni, ogni nodo rappresenta l'insieme degli attributi soluzione, sto considerando i gruppi:



pubblicherai i dati che corrispondono al cluster che hai fatto. Dal padre verso un figlio significa dividere un insieme, il passo dal figlio al padre è mettere insieme due insiemi. Quello che si fa è dalla radice (che vuol dire la cosa più generalizzata possibile) vai giù con un BTF se arrivi ad un nodo in cui la proprietà non è soddisfatta non ho bisogno di scendere ulteriormente e posso tagliare quel pezzo dell'albero. L'albero è un albero di copertura della gerarchia che avremmo, la radice è quello di un solo insieme che ce li ha tutti. Se guardiamo come hanno costruito l'albero è sempre vero che da un padre ad uno dei figli si ottiene facendo un taglio di uno dei suoi insiemi. Viceversa da un nodo qualsiasi andando su al padre si trova unendo due dei suoi insiemi, questo rappresenta tutte le possibili soluzioni che posso avere.

Percorro l'albero delle soluzioni, se scendo e vedo che un nodo non soddisfa k -anonymity posso eliminare tutto quello che viene giù dal nodo, posso fare il pruning di quella parte dell'albero. Vuol dire cerca di non analizzare tutte le soluzioni possibili ma tagliare lo spazio di ricerca quando sto percorrendo un cammino. Sfrutta la proprietà di monotonicità.

Incognito Algorithm

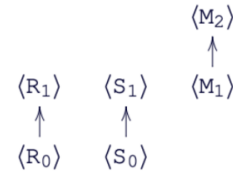
Vuoi raggiungere k -anonymity, ciascuna combinazione di valori deve apparire almeno k volte. Te lo impongo sulla tupla dell'identifier. Condizione necessaria perchè tu abbia le k -occorrenze su una ennupla di valori è che tu abbia le k occorrenze anche nella tupla.

Per rispettare la k-anonymity a livello di tupla devi rispettarla a livello di tutte le sotto-tuple. In modo simile al pruning, se notiamo che un attributo a livello 0 non soddisfa k-anonymity qualsiasi tabella che tiene quell'attributo a livello 0 non soddisferà k-anonymity, quindi quando costruisce il reticolo non lo taglia e non tiene quello.

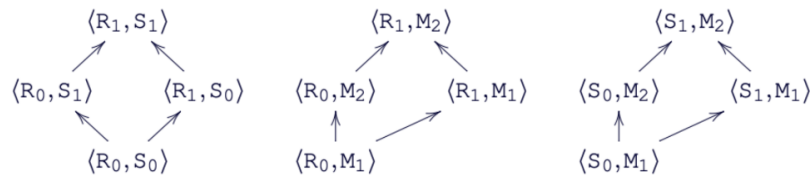
Al secondo passo guardo a coppie e butto via tutte le combinazioni di attributi che non soddisfano. Vado avanti così fino all'i-esimo passo.

Race	Sex	Marital status
asian	F	divorced
asian	F	divorced
asian	F	married
asian	M	married
asian	M	married
black	F	single
black	F	single
white	F	single
white	F	widow

Iteration 1

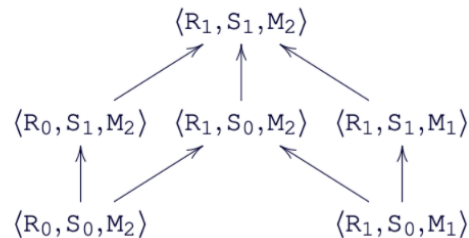


Iteration 2



Race	Sex	Marital status
asian	F	divorced
asian	F	divorced
asian	F	married
asian	M	married
asian	M	married
black	F	single
black	F	single
white	F	single
white	F	widow

Iteration 3



2.3.6 Algoritmi Euristici

Quelli visti prima sono algoritmi esatti np-completi, ovviamente esistono algoritmi euristici che non danno la soluzione esatta ma la approssimano.

Mondrian Multidimensionali

Sfruttano la rappresentazione dei dati su uno spazio multidimensionale e poi sfruttano algoritmi di taglio sullo spazio multidimensionale. Un attributo del quasi-identifier rappresenta una dimensione, se abbiamo una coppia avremo un piano, una tripla un piano tridimensionale, tre attributi uno spazio a tre dimensioni e così via. Rappresentano i dati

che abbiamo in forma tabellare traslati su uno spazio multidimensionale, la rappresentazione è che ciascuna tupla corrisponde ad un punto nello spazio. Oltre al punto devo scrivere anche il numero di occorrenze. Metto i valori in fila, in modo simile a k-optimizer, c'è un ordinamento implicito dei valori. Adesso facciamo runnare gli altri algoritmi in cui ad ogni passo scegliamo una dimensione in cui fare il taglio. Ci si creano due sotto spazi e di quegli scegliamo uno da tagliare e si creano due sotto spazi, fino a che non troviamo uno spazio che non possiamo tagliare perchè scenderebbe sotto le k-tuple che vorremmo.

è uno dei più utilizzati perchè riesce a produrre una soluzione efficiente ed ha delle flessibilità:

- Flessibile rispetto a quanti attributi hai nel quasi identifier
- Può fare attribute generalization e self generalization
- Due strategie di partizionamento:
 - Stricted: non overlapping
 - Relaxed: potentially overlapping

La flessibilità è buona, rimane non carino il fatto di dover mettere i valori in ordine.

wished $k=3$

Race	ZIP
asian or white	9414*
asian or white	9414*
asian	94139
asian	94139
asian	94139
black or white	9413*
black or white	9413*
black or white	9413*
asian or white	9414*

GT

asian	3	1	1
white	1	1	
black	1	1	
	94138	94139	94141 94142

Algoritmi di approssimazione per generalizzazione livello Cella

Non sono algoritmi esatti, ma che mi portano ad una soluzione che non costa più di un tot rispetto all'ottimo, 2 approx non mi porta ad un costo due volte superiore all'ottimo, il costo è la perdita di utilità.

2.3.7 k-anonymity revisited

Se andiamo a livello di cella vogliamo bloccare il record linkage, vuol dire che male che vada il mondo fuori (non confidenziale) è composto da tutti e soli i nostri dati, allora devo vedere qual'è la capacità di re-identificazione di qualcuno che ha la nostra tabella e i dati esterni così vediamo che ogni generalizzazione di tupla può portare ad una identificazione tra n elementi della tabella originale.

Race	ZIP	Race	ZIP
white	94138	person	9413*
black	94139	person	9413*
asian	94141	asian	9414*
asian	94141	asian	9414*
asian	94142	asian	9414*
PT		2-anonymity	

Race	ZIP	Race	ZIP	Race	ZIP
person	9413*	person	9413*	person	9413*
person	9413*	person	9413*	person	9413*
asian	94141	asian	9414*	asian	94141
asian	9414*	asian	9414*	asian	94141
asian	9414*	asian	94142	asian	9414*
2-anonymity (revisited)		no 2-anonymity		no 2-anonymity	

Quindi quello che devo garantire è che per ogni sequenza di valori nella tabella originale devono esserci almeno k-tuple nella tabella generalizzate e viceversa, per ogni tupla nella tabella generalizzata devono esserci k-tuple nella tabella originale.

Dal punto di vista teorico la prima tabella in basso è più bella, però quella generica di k-anonymity è quella che viene utilizzata. Questo perchè la prima tabella con le k-occorrenze è più generale e può essere utilizzata senza conoscere i dati originali (posso dire quanto sono protetti i dati guardando solo la tabella sanitizzata), mentre con le altre generalizzazioni devo conoscere i dati originali.

2.4 Divulgazione degli attributi

k-anonymity è vulnerabile ad alcuni attacchi che ora approfondiremo.

2.4.1 Problema di omogeneità

I gruppi uguali sono sensibili agli attacchi di homogeneity attack, se fai parte di un gruppo uguale, se so che sei lì dentro è vero che non ti identifico ma so sicuramente la caratteristica in comune che hai con gli altri (ad esempio la malattia comune).

2.4.2 Background knowledge

Stiamo facendo protezione con un'assunzione molto forte, quello che puoi sfruttare per fare identificazione è fare linking con gli altri gruppi di quasi identifing. Sto assumendo che puoi vedere altre tabelle che hanno gruppi in comune con la mia, la realtà è che fuori possiamo sapere tante cose, qualcuno potrebbe avere informazioni ad esempio a livello di istanza. Quindi la background knowledge è un'informazione che inietta perchè so qualcosa di qualcuno e mi permette di ridurre la generalizzazione.

2.4.3 l-diversity

Se vuoi evitare problemi di omogeneità o se vuoi rendere più difficile eventuali attacchi che sfruttano la conoscenza esterna, quello che dovresti fare quando definisci dei gruppetti dovresti farli diversi dal punto di vista delle informazioni sensibili, devo dare incertezza dal punto di vista dell'informazione sensibile. Quando fai i blocchi q-block per garantire k-anonymity verifica che contengano almeno l valori diversi per gli attributi sensibili.

Quindi una tabella la chiamo l -diversa se ha l valori diversi in ciascun gruppetto, notiamo che l non può essere più grande di k (se ho 10 persone e una malattia per persona non potrò avere più di 10 malattie diverse). Così facendo l'attacco di omogeneità non è più possibile, mentre il background knowledge è più difficile.

Potresti avere ancora attacchi basati sulla similarità dei valori e attacchi di skewness.

2.4.4 Skewness attack

Con un rilascio potremmo aumentare la confidenza di conoscenza di alcune caratteristiche sensibili. Se mettiamo in un gruppetto, ad esempio, in cui la maggior parte della gente ha una caratteristica sensibile possiamo inferire qualcosa (ad esempio 10 persone di cui 9 hanno votato la stessa cosa, la probabilità che tu abbia votato quella cosa schizza al 90%). Formalmente l'attacco di skewness si verifica quando la distribuzione in un blocco q è diversa dalla distribuzione nella popolazione originale, ad esempio il 20% della popolazione mondiale soffre di diabete ma nel blocco q il 75% ne soffre.

2.4.5 Similarity attack

Il problema è nasconderti in qualcosa di troppo simile, se non è zuppa è pan bagnato. Nell'esempio abbiamo visto un gruppetto con due con ulcera allo stomaco e uno con la gastrite, sono entrambe malattie allo stomaco. Quando applichi la diversity non guardarlo solo sulla diversità sintattica dei valori, ma anche su una diversità semantica.

2.4.6 Group closeness

I q-blocchi me li fai con una proprietà di t -closeness, in cui t mi dice quanto sono vicino rispetto alla distribuzione nel mondo reale (3 diabetici su 4 non è vicino al 25% di persone diabetiche nel mondo). La buona notizia è che anche la t -closeness è monotonica. Una tabella rispetta t -closeness se rispetta k -anonimity e l -diversity, ma anche se rispetta il grado t . È il modo di misurare l'informazione data dal rilascio.

Può essere carina dal punto di vista teorico, ma la soppressione è talmente tanta che non si fa.

2.4.7 External Knowledge modeling

Potresti avere informazioni a livello di istanza da poter iniettare per prendere informazioni:

- Puoi cercare informazioni di un target
- Puoi iniettare informazioni che sai su altri

- Puoi iniettare informazioni che sai avere in comune su un gruppo, individui che condividono con l'individuo target certi valori. Ad esempio uffici condivisi con telefono singolo

2.4.8 Rilasci multipli

Il problema è quando mi trovo a rilasciare più volte un'informazione simile per un individuo, tutti i mesi l'università rilascia le informazioni sugli studenti. La cosa più banale che può capitare è un attacco di intersezione. Innanzitutto una cosa a cui devo stare attento è cambiare l'attributo che generalizzo in diversi rilasci (ad esempio in uno la data di nascita, in uno il luogo di nascita).

Sei hai una sanitizzazione e quello che viene sanitizzato dipende solo da un valore possiamo permetterci di avere i dati grezzi ed ogni volta che dobbiamo rilasciare produciamo la versione per il rilascio, ad esempio rimuovere le ultime cifre da un numero di telefono. Non possiamo permetterci di fare questa cosa se abbiamo una tecnica di protezione non deterministica e puntuale, come tutto quello che abbiamo visto oggi, produrre i dati che rispettino la k -anonymity non è deterministico.

Quindi quando faccio questi rilasci multipli, che possono essere sullo stesso dataset o su dataset con tanta componente in comune devo stare attento ad attacchi di intersezione.

2.4.9 m-invariance

Se fai rilasci multipli su un dataset, devi avere almeno m tuple, nessun valore sensibile appare più di una volta in ogni gruppo e per ogni tupla i valori sensibili che vedi ogni volta sono sempre gli stessi. È facile dirlo in teoria ma in pratica nel mondo reale è molto difficile.

2.4.10 Extended scenarios

Le metriche primarie sono k -anonymity, l -diversity e t -closeness. Esistono anche delle variazioni:

- Se avessi più tuple per paziente il vincolo di cardinalità (essere confuso con k pazienti) potrei dargli un minimo di cardinalità perchè può aiutare a confondere di più ma espone anche ad un problema di inferenza da aggregazione orizzontale
- Assunzione di avere una tabella universale, però tipicamente abbiamo tante tabelle da rilasciare e le tabelle non sono indipendenti e non essere indipendente non vuole dire avere attributi in comune ma anche avere attributi che sono in qualche modo legati uno all'altro
- Avere più quasi-identifier, anche qui possiamo dargli la tara, possiamo fare l'unione degli attributi
- Non sappiamo i quasi-identifiers, è più naturale di quello che si pensa, andiamo ad inferire dai dati quello che è un quasi-identifier

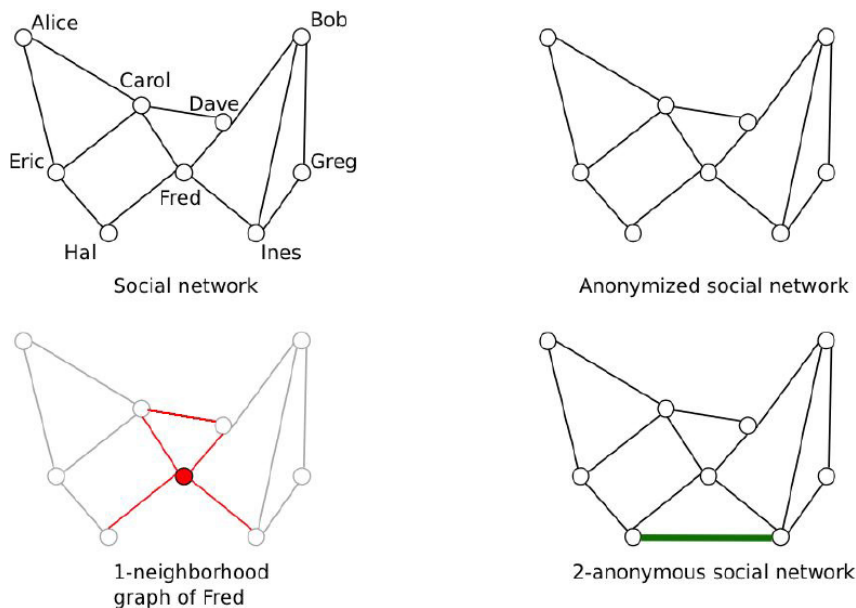
- Rilascio di data stream, immaginiamo di voler applicare una protezione di k-anonymity ad esempio sui log di browsing o di sensori e camere in giro per la città, abbiamo un flusso di dati. La risposta più semplice è rilasciare periodicamente, non il flusso.
- Preferenze di privacy a grana più fine, ad esempio applicazioni di queste in un contesto in cui il data subject salta su a cittadino di primo livello dell'ecosistema, ad esempio dati che arrivano da smartwatch

2.5 k-anonymity in vari scenari

Può essere applicata in vari scenari:

- social networks
- data mining
- location data

2.5.1 Social Networks



Prendiamo qualsiasi applicazione in cui abbiamo una rete sociale, se ho il grafo originale e una possibile versione anonima potremmo risalire all'identità dal numero di connessioni che ha quel nodo. Si chiama neighborhood attack. Se sei l'unico con n connessioni allora sei esposto, se ce ne sono tanti con n connessioni sei tranquillo. Puoi fare questo ragionamento anche a distanza maggiore di 1, quanti amici hai che hanno n amici?

Puoi fare due cose:

- Aggiungere false connessioni
- Togliere connessioni

Per trasformare un grafo in cui hai k nodi che sembrano uguali al numero n di connessioni.

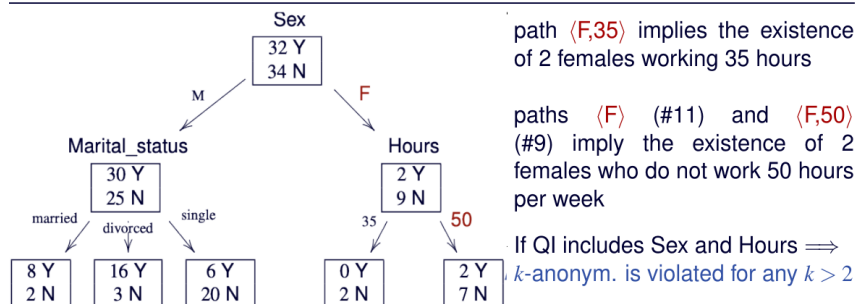
2.5.2 Data mining

In un contesto di data mining la cosa più semplice posso pensare di vedere regole di correlazione tra i dati o costruire alberi di decisione che possono servire per classificare altre informazioni. Ci sono due cose che misuro:

- Supporto
- Confidenza: quante volte divorziato e maschio compaiono insieme rispetto al totale dei divorziati

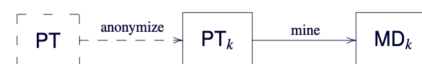
Posso anche osservare dei dati, che potrei chiamare training set, per imparare come comportarmi per osservazioni future che verranno, ad esempio potrei trovare un'occorrenza per cui se fai determinate cose puoi essere a rischio di malattie.

<i>Marital_status</i>	<i>Sex</i>	<i>Hours</i>	<i>#tuples (Hyp. values)</i>
divorced	M	35	2 (0Y, 2N)
divorced	M	40	17 (16Y, 1N)
divorced	F	35	2 (0Y, 2N)
married	M	35	10 (8Y, 2N)
married	F	50	9 (2Y, 7N)
single	M	40	26 (6Y, 20N)

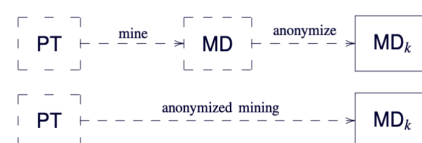


Come anonimizzo i dati in caso di data mining? posso avere un contesto in cui anonimizzo o posso fare il mining e poi anonimizzo o fare le cose insieme. Dal punto di vista delle assunzioni di sicurezza possiamo vedere dove è il trust boundary, perchè nel primo caso abbiamo i dati, li anonimizziamo e quando li abbiamo puliti a questo punto fai il mining che vuoi, tanto ormai è tutto sanificato, ma negli altri casi ho sempre dentro informazione sensibile, quindi il mining lo può fare solo chi è autorizzato a vedere i dati.

Anonymize-and-Mine



Mine-and-Anonymize



2.5.3 Location base service

Il contesto è ad esempio in un servizio di locazione, osservi degli eventi e vogliamo proteggere l'identità di chi ha fatto quell'evento (una richiesta di servizio) in un contesto in cui quello che succede è che quello esposto dalla persona è la locazione in cui è.

Dovremmo generalizzare la locazione in modo da avere più gente lì, offuschiamo la locazione allo stesso modo in cui prima generalizzavamo altri attributi. La location equivale al nostro quasi-identifier.

Esiste un altro contesto in cui la location equivale a quella che era la nostra informazione sensibile, ad esempio sto facendo una richiesta di servizio e non mi interessa che sai che sono io, ma non mi piace che tu sappia in che location sono io.

Un'altra cosa è proteggere il cammino, perchè magari è quello che mi identifica, se prendiamo il punto casa al lavoro siamo noi sicuro. Già se prendiamo dalla stazione del pullman ad un'altra abbiamo più persone. Quindi è proteggere il cammino perchè può essere sfruttato per capire chi ci sta dietro.

2.6 Re-identificazione

Qualsiasi bit di informazione in comune su diverse sorgenti mi espone informazioni sensibili, ci sono due esempi:

- AOL: America Online era un servizio per fare le query sul web. Nel 2006 per fare vedere un'apertura rispetto all'accademia ha rilasciato disponibili 20 milioni di query di ricerca di 650k utenti su 3 mesi di attività, hanno rimosso qualsiasi informazione identificante (incluso il nickname) e hanno rimosso anche l'indirizzo IP e hanno rimpiazzato lo user name con degli alias che permettevano di fare link tra ricerche. Dei tipi del new york time sono riusciti a re-identificare la persona sulla base di cose che apparivano. Il problema è che hanno fatto pseudo-anonimizzazione statica, perchè lo pseudonimo mi rimane attaccato, permetti di linkare informazione sulla stessa persona.
- Netflix: avevano deciso di dare 1 milione di dollari per chi riusciva a fare un algoritmo di raccomandazione che funzionava il 10% meglio del loro. Il problema è che ha rilasciato 100 milioni di record su come mezzo milione di utenti aveva trattato i film, per ogni record ha rilasciato il numero di stelline e la data. Ha fatto campionamento, rumore (ha perturbato i dati, ma non troppo, perchè gli serviva capire come funzionava l'algoritmo), user name rimossi e un alias messo al posto dello user name chiaramente come AOL non uno pseudonimo dinamico ma statico. Hanno visto che i dati non erano proprio anonimi, le osservabili che avevano erano abbastanza peculiari per fare spiccare gli individui. Diventano poi riconoscibili perchè ad esempio se l'utente ha fatto lo stesso rating su IMDb riuscivano a riconoscere l'utente (correlano record con fonti esterne)

Vediamo un altro paio di esempi da menzionare. Il primo è il caso JetBlue, per il GDPR c'è sempre il purpose of use e di regolamentazione del secondary use. Quando rilascio le informazioni a qualcuno, se sono informazioni sensibili. Il rilascio dei dati in prima battuta è per lo scopo primario (carta di credito per fare l'acquisto) da lì in poi

tutti gli altri scopi sono secondari. In questo caso si sta parlando di un contraente della compagnia aerea per scopi personali di marketing, questo per dire che se do i dati a qualcuno c'è sempre di mezzo il secondary use, potrei trovarmi nel contesto di limitare i secondary use.

Un'altra menzione è quella di Strava, se tutti i militari americani vanno a fare jogging e tutti usano Strava per tracciare il percorso non è così difficile risalire alla base, perchè tutti corrono intorno alla base.

2.7 Privacy semantica

Tutta questa scia ha fatto dire che forse dovremmo fare qualcosa di diverso, stiamo cercando di rilasciare dei microdati però cercando di implementare una misura di protezione che arriva fino ad un certo punto, quindi tutte le misure di protezione sintattiche (k-anonymity...) che catturano il grado di protezione che dai agli individui rispetto a proteggere anonimato o info sensibili arrivano fino ad un certo punto. Ha fatto nascere la definizione di privacy semantica, i dati che ti restituisco ci metto dentro del rumore.

Questa è la differential privacy, cambia filosofia e cerca di rispondere a delle richieste di rilascio dove la promessa agli individui è che il rilascio è indipendente dal fatto che ho rilasciato i tuoi dati o no, non rilascio dati sensibili e dal rilascio nessuno riesce a capire se la tua tupla è presente o no. È una proprietà sul rilascio.

È più difficile da misurare perchè è basata su un parametro di protezione (differential privacy), informalmente dice che il risultato di una query è sostanzialmente lo stesso a prescindere o meno del record di un certo individuo. La probabilità di avere una certa risposta nel dataset completo o in quello senza la tupla è la stessa. Funziona con un budget, quando lo esaurisci significa che hai rilasciato troppo e devi buttare il dataset.

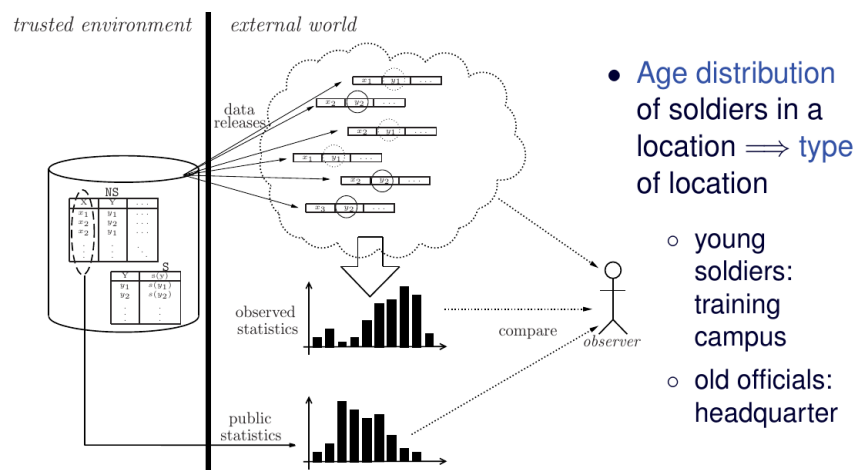
È nato in un contesto di statistical DBMS (fai query e ci ciocco dentro del rumore) è stata poi estesa a scenari simili a quelli chiamati statistical data (filosofia sempre del ciocco del rumore). Più succede che il mio dataset ha valori peculiari più sarà il rumore che dovrò aggiungere. Lo faccio aggiungendo rumore, significa che non sto rilasciando più microdati e dati corretti. Abbiamo anche uno scenario di differential privacy che può operare a livello globale o locale, la differenza è in base allo scenario in cui sei, se sei in un contesto in cui c'è una data authority che sanitizza i dati per il suo rilascio però chi ha in mano i dati è fidato siamo in globale, invece in un contesto locale chi colleziona i dati non è fidato.

2.7.1 Differenze con k-anonymity

- k-anonymity: fa una certa promessa e mantiene quella, di essere confuso con un gruppo
- differential privacy: ha delle garanzie di protezioni più forti ma funziona solo con delle query

2.8 Sensitive Value Distributions

Ho un contesto in cui le tuple non sono sensibili, però una collezione di tuple può fare leakage di informazioni sensibili che tra l'altro non sono neanche rappresentate dai dati. Qui si trattava di avere a che fare con record medici di militari, rilascio ad ogni militare il suo record, poi ho anche il fatto che il personale può rivolgersi tutto ad uno stesso player fuori per certi servizi, mentre i record di ciascun militare non sono sensibili la loro visione completa lo è. Il leakage che fa lo fa di una cosa che neanche c'è nel record, che è il tipo di locazione a cui il militare afferisce (training campus, head quarter), la distribuzione delle età fa inferire il tipo di campus da cui arrivano i dati.



Questo giusto per dire che esistono problemi strani lì fuori.

2.9 Dati genomici

Sono sensibili perchè identificano il proprietario, fanno leakage con un po' di informazioni (razza, età, abitudini, malattie o predisposizioni a malattie). Non espone la persona ma anche quelle a lui legate.

Il progetto 1000 genomes costruiva dataset di dati genomici, è successa una re-identificazione di 5 uomini e da loro anche i familiari per comunanza di dati genomici

2.10 Sensitive inference from data mining

Possiamo fare target identifier, ad esempio se una donna giovane compra determinati prodotti è probabile che gli nasca un bambino. C'è una profilazione che è al di là di quello che ci si può immaginare.

2.11 Social Media

In questo caso siamo nel caso in cui sono gli utenti stessi che spifferano dati sensibili. Se si guarda quanto viene rilevato dal profiling sui social media su cose che sembrano innocenti sono un sacco.

A volte il punto debole di tutto ciò è l'utente, bisogna stare attenti che anche se pensiamo di condividere solo con gli amici Facebook sa tutto. Inoltre si è visto che non è vero che condividiamo solo con gli amici, molto spesso nella rete non abbiamo solo amici veri, ma anche gente che non conosciamo o bot.

Se guardiamo rispetto al social che usiamo le informazioni che sa su di noi sono un sacco. Le informazioni che vengono collezionate e inferite da cose che sembrano innocenti sono tantissime.

2.11.1 Cambridge analitica

Il problema è stato dei leak di informazioni prese da applicazioni che chiedevano il permesso di usare i nostri dati, c'era un quiz di personalità installata da 330k utenti Facebook che collezionava anche dati degli amici, che sono stati condivisi con Cambridge analitica per profilare gli utenti.

Dietro avevano un modello OCEAN:

- Openness: ti piace fare nuove esperienze?
- Conscientiousness: preferisci i piani e l'ordine?
- Extraversion: quanto sei social?
- Agreeableness: dai valore ai bisogni degli altri e della società?
- Neuroticism: quanto tendi a preoccuparti?

Facevano un modello psicologico di quello con cui stavo parlando e in base a questo modello gli proponevo informazioni. Un altro modo in cui fanno profilazione è attraverso i quiz online.

I più scettici dicono che nel futuro vivremo tutti la stessa realtà ma raccontata per come piace a noi.

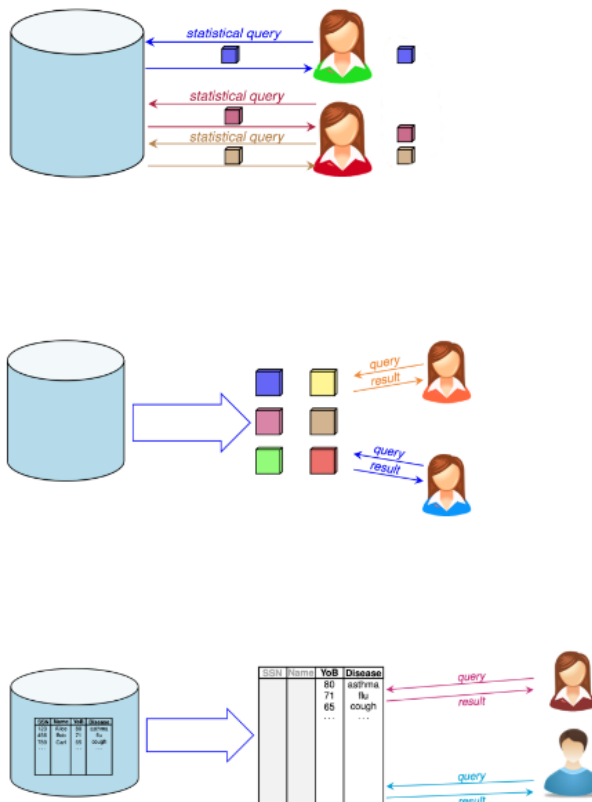
Un altro grosso problema ovviamente è il riconoscimento facciale, se sono sistemi automatici che fanno riconoscimento facciale è grave per la nostra privacy (e siamo noi che mettiamo le informazioni online per fare il training di questi modelli).

Chapter 3

Protezione Macrodati e Microdati

Quando parliamo di rilascio eravamo in 3 scenari, nel modulo di prima:

- Ho uno statistical DBMS, risponde solo a query statistiche. Il DBMS deve essere trusted. Abbiamo problemi di collusione e di observation window (come tratto la gente diversa che mi fa domande e come tratto i dati che ti ho già dato).
- Ho una produzione offline di macrodati, materializzazione di query statistiche che rilascio. Anche qui il DBMS deve essere trusted, però lavora offline.
- Rilascio i microdati



Prima abbiamo visto problematiche di microdati dove la preoccupazione è l'anonimità dei soggetti nei microdati per poter andare ad estendere la protezione dell'attribuito.

Adesso facciamo un passo indietro e vediamo una carrellata di tecniche di base che si fanno nel campo di statistical DBMS e macrodati.

3.1 Statistical data dissemination

Quando è nata k-anonymity il problema è che quello che c'è in giro erano i microdati, non si riuscivano a proteggerli quindi si è guardato ai contesti, molto spesso nei contesti bastavano dati aggregati. Il problema non è che avere dato statistico sei tranquillo, può spifferare info:

- Trovare nei dati
- Linkage con dati che hai già rilasciato

- Combinazioni di dati che hai rilasciato con info che può essere trovata da altre parti

Siamo in due contesti:

- Statistical DBMS: rilascia solo dati da query statistiche
- Statistical Data: si computano le statistiche offline e si pubblicano le statistiche, per interfacciarti con i data consumer non ti serve più essere in un ambiente trusted

3.2 Statistical DBMS

Risponde a query statistiche, il DBMS che si interfaccia con l'utente deve essere trusted.

Quindi ad esempio supponiamo di avere una tabella così, questi sono i dati grezzi :

Name	Sex	Major	Class	Income
Allen	Female	CS	1980	68k
Baker	Female	EE	1980	50k
Cook	Male	EE	1978	70k
Davis	Female	CS	1978	80k
Evans	Male	EE	1981	60k
Frank	Male	CS	1978	76k
Good	Male	CS	1981	64k
Hall	Male	EE	1978	60k
Iles	Male	CS	1979	70k

Non potrei rispondere una domanda del tipo: dammi gli income delle donne laureate in EE, c'è un solo record. Sostanzialmente se mi colpisci un target set troppo piccolo non ti rispondo.

Un altro esempio è: dammi la somma dei ricavi degli individui con un major in EE. Questa risposta posso darla perchè è sopra la dimensione del target set che ho come riferimento per la privacy. La query non è sensibile.

Dammi la somma degli income dei maschi con un major in EE, anche questa posso rilasciarla perchè non è sensibile. Qui però c'è la storia delle osservazioni, se prima mi chiedi quella di tutti gli EE e te la dico poi mi chiedi quella dei maschi e te la dico, poi se fai la differenza trovi il dato sensibile della donna. Questo è l'esempio dell'observation window, devo tenere traccia di quello che mi chiedi. È quello che fa differential privacy togliendo budget tutte le volte, mentre nelle soluzioni classiche si tiene in memoria quello che ti ho già dato.

3.3 Macrodati

Vuol dire che abbiamo query con i dati grezzi, produciamo dei dati aggregati per il rilascio, abbiamo l'ambiente trusted che è il dbms e gli utenti si interfacciano direttamente con i dati quindi sono in ambiente untrusted.

Abbiamo due categorie di dati:

- Tabelle di frequenza o conteggio: quante persone hanno una certa proprietà

- Tabelle di grandezza: ciascuna cella è un aggregato rispetto a qualche entità

Nella protezione di dati statistici se devo produrre dati aggregati la prima tecnica che attuo è il campionamento, calcolo le statistiche non su tutti i dati ma su un campione. Dopo di che il problema è se questa computazione crea problemi per privacy, devi identificare se ci sono delle celle sensibili che non possiamo rilasciare, le possiamo identificare sulla base di:

- Regole speciali
- Regole di soglia, al di sotto di n persone no

Una volta che ho capito che nella tabella ho delle celle sensibili che non posso rilasciare cosa faccio? devi operare nella tabella andando a fare qualche cambiamento (sopprimere i dati) per coprire queste celle.

3.3.1 Campionamento

Considero solo un sottoinsieme dei dati che ho. Quindi quando computo la statistica peserò i campioni sulla base del peso che hanno nella popolazione generale, si pubblicano i pesi o no? in realtà sarebbe meglio di no. Tutte le volte che abbiamo regole di questo tipo se pubblico i parametri mi espongo ad inferenza, però chiaramente devo utilizzare dei pesi per pesare il campione rispetto al campione sulla popolazione che rappresenta per garantire l'accuratezza statistica.

3.3.2 Regole

Le regole speciali definiscono il livello di dettaglio sotto al quale non posso andare. É speciale perchè non c'è una regola definita e dipende dal tipo di dati che sto trattando. Un esempio è che non pubblichi celle che siano uguali al totale o non pubblichi celle che permettano di inferire l'età delle persone in un intervallo di 5 anni.

Ad esempio ho questa tabella e non permetto di inferire un income con 5k di intervallo:

Dept	Income						Total
	[0-21)	[21-23)	[23-25)	[25-27)	[27-29)	29+	
Dept ₁	2	4	18	20	7	1	52
Dept ₂	-	-	7	9	-	-	16
Dept ₃	-	6	30	15	4	-	55
Dept ₄	-	-	2	-	-	-	2

le celle problematiche sono la cella 2 e le celle con 7 e 9.

Vediamo un altro esempio di special rule, quando pubblichi informazioni sanitarie devi rimuovere informazioni identificanti, devi generalizzare in modo che ciascuna locazione abbia almeno 20k residenti e anche le date di nascita devi generalizzarle. Cosa chiamiamo regole di soglia, quella che mi dice ad esempio, se una cella ha meno di 5 rispondenti non puoi rilasciarla.

Company	Education level				Total
	Low	Medium	High	Very High	
Alfa	15	1	3	1	20
Beta	20	10	10	15	55
Gamma	3	10	10	2	25
Delta	12	14	7	2	35
Total	50	35	30	20	135

3.3.3 Protezione celle

Adesso veniamo a cosa faccio per proteggere queste celle, prendiamo questa tabella:

La regola di soglia dice che sotto i 5 rispondenti non pubblici, vediamo colorate le celle che mi creano problemi.

Oppure potresti ristrutturare la tabella, vediamo un esempio:

Dept	Income						Total
	[0-21)	[21-23)	[23-25)	[25-27)	[27-29)	29+	
Dept ₁	2	4	18	20	7	1	52
Dept ₂	-	-	7	9	-	-	16
Dept ₃	-	6	30	15	4	-	55
Dept ₄	-	-	2	-	-	-	2

cosa significa ristrutturare la tabella? in questo caso potremmo combinare i dipartimenti, sostanzialmente mi va a fare il roll-up, raggruppa in una dimensione o nell'altra per soddisfare le regole.

Soppressione

La soppressione vuol dire togliere delle celle, non le rilasciamo. Se una cella la identifichiamo come sensibile la cancelliamo e non la rilasciamo.

Vediamo un esempio, se la soglia è 5 troviamo le celle sensibili:

Company	Education level				Total
	Low	Medium	High	Very High	
Alpha	15	1	3	1	20
Beta	20	10	10	15	55
Gamma	3	10	10	2	25
Delta	12	14	7	2	35
Total	50	35	30	20	135

Le celle sensibili devo eliminarle, siamo tranquilli? no, perchè sapendo gli altri dati e la somma posso risalire ai dati che ho soppresso. Ad esempio se volessi risalire al valore $< Alpha, Medium >$ che abbiamo soppresso: $35 = D_1 + 10 + 10 + 14 \rightarrow D_1 = 1$ Quindi la soppressione che ho fatto, quella primaria, non basta.

Quindi oltre a questa soppressione dobbiamo avere un'altra soppressione (secondaria) che aiuta a coprire le inferenze che posso fare per ricostruire i valori soppressi, il problema

è cosa sopprimere (si usano tecniche di programmazione lineare). Quello che facciamo nell'esempio è di sopprimere un valore addizionale per ogni riga e colonna che ha un valore sensibile soppresso.

Rounding

Significa arrotondare i valori, i valori che pubblici non sono i valori veri. Può essere:

- Random: decido in modo casuale se una cella deve essere arrotondata sia in eccesso che difetto, potrei essere in una situazione in cui la somma non torni
- Controllato: per assicurarsi che le somme per righe e colonne siano corrette

Quello che faccio è pubblicare le celle in modo tale che siano multiple per il mio valore di soglia.

Company	Education level				Total
	Low	Medium	High	Very High	
Alfa	15	1	3	1	20
Beta	20	10	10	15	55
Gamma	3	10	10	2	25
Delta	12	14	7	2	35
Total	50	35	30	20	135

Company	Education level (random rounding)				Total
	Low	Medium	High	Very High	
Alfa	15	*0	*0	*0	20
Beta	20	10	10	15	55
Gamma	*5	10	10	*0	25
Delta	*15	*15	*10	*0	35
Total	50	35	30	20	135

In questo esempio ho arrotondato 12, 14 e 7 perchè non erano multipli di 5, è random perchè le somme non tornano.

Fare la cosa in modo controllato significa che quando faccio gli arrotondamenti dei valori (tutto quello sensibili e quello che non è multiplo della soglia sensibile) lo facciamo in modo che tutti conti tornino:

Company	Education level				Total
	Low	Medium	High	Very High	
Alfa	15	1	3	1	20
Beta	20	10	10	15	55
Gamma	3	10	10	2	25
Delta	12	14	7	2	35
Total	50	35	30	20	135

Company	Education level (controlled rounding)				Total
	Low	Medium	High	Very High	
Alfa	15	*0	*5	*0	20
Beta	20	10	10	15	55
Gamma	*5	10	10	*0	25
Delta	*10	*15	*5	*5	35
Total	50	35	30	20	135

Confidentiality edit

Significa andare a cambiare i dati, non cambiamo le statistiche che abbiamo calcolato (non facciamo una ristrutturazione o cambiamo il valori), ma andiamo a cambiare i dati grezzi su cui le statistiche sono calcolate.

Lo faccio sostanzialmente con uno swapping dei dati:

- Prendi un campione dei microdati
- Trova un match per alcune delle tue tuple in un insieme nella popolazione di un altro insieme
- Scambia i dati tra i due insiemi

Quando hai dati piccoli magari applichi più swap. Questi sono microdati curati per avere protezione dei macrodati che escono.

Quando paciughiamo i dati in questo modo otteniamo una tabella di aggregazione che contiene del rumore. Quello che mi garantisce protezione è che i dati che sto pubblicando non sono reali ma sono rumorosi.

3.3.4 Protection of tables of magnited data

Il problema del campionamento qui sono gli outlier. Il problema qui è come identifico le celle sensibili, stiamo andando a vedere se è esposta una tua proprietà, ad esempio se hai la somma del fatturato se c'è qualcuno che ha il 90% chiaramente è esposto.

p-percent

Quando hai tabelle di grandezza, ad esempio hai la somma del fatturato, a questo punto puoi dire che una certa quantità x è la somma del fatturato, hai rappresentate in questa grandezza delle aziende qui vuoi proteggere la singola azienda. Chi può violare questa cosa sono gli esterni sia chi partecipa, perchè può togliere la sua quantità dal totale.

L'identificazione di una cella sensibile è se chi guarda i dati può inferire al disotto di un certo livello di incertezza l'informazione che è di qualcuno.

Esiste anche il problema di coalizione, se la gente si mette insieme cosa impara? Formalmente diciamo che una cella è protetta, e p è il grado di protezione, se la somma dei valori per la dimensione della colazione +2 fino al numero totale dei rispondenti per la somma della grandezza è maggiore uguale della p percento del rispondente che ha valore maggiore:

$$\sum_{i=c+2}^N x_i \geq \frac{p}{100} x_1$$

Dati questi dati:

- Alice: 100k
- Bob: 80k
- Carol 30k
- David: 20k
- Eve: 10k
- Frank: 3k item ...

Ad esempio qual'è la colazione di 3 persone che può stimare meglio lo stipendio di Alice? i tre più ricchi; Bob, Carol, David:

$$\sum_{i=3+2}^N x_i \geq \frac{p}{100} 100$$

Quindi se mi specifichi un p posso dirti se puoi uscire con quei dati o meno, p ci dice che c'è un'incertezza di $p\%$ sul valore. Formalmente una cella è protetta per ogni p tale per cui:

$$p \leq Cell - (Alice + Bob + Carol + David)$$

pq

Più o meno la stessa cosa di prima, andare a vedere qual'è la confidenza con cui una coalizione può inferire valori sui partecipanti, ancora una volta si guarda il caso peggiore, ovvero il partecipante con valore più alto. Possiamo però assumere una conoscenza pregressa, 1 ci dice un intervallo di incertezza di conoscenza che la coalizione già sa.

Quindi abbiamo la stessa cosa di prima, il modo in cui cambia è avere il q su 100, sostanzialmente quando q è 100 è la stessa cosa:

$$\frac{q}{100} \sum_{i=c+2}^N x_i \geq \frac{p}{100} x_1$$

Se supponiamo $q=80$ che rappresenta l'accuratezza che un rispondente ha per stimare un dato ancora prima che pubblico i dati, l'outcome di Alice può essere tra 20% e 80%.

Il ragionamento è esattamente lo stesso di prima, salta fuori un p diverso perchè è tarato sul q :

$$\frac{80}{100} \sum_{i=3+2}^N x_i \geq \frac{p}{100} 100$$

ora ci viene fuori un $p \leq 16$. Ci dice solo che la soglia che troviamo è sotto p va bene, altrimenti non va bene.

$$Cell - (Alice + Bob + Carol + David) \leq \frac{p}{0.80}$$

(n,k)

Quando una cella è sensibile? quando abbiamo 1 o poche parti che contribuiscono tanto. n ci dice se stiamo proteggendo l'esposizione di 1 o di un piccolo insieme, k è quello che consideri come soglia sensibile ovvero quanto contribuiscono al totale dei dati. Ad esempio bob e alice contribuiscono al 72%, se mi dici una somma di (2,70) non va bene, ma se è (2,80) va bene.

Soppressione

Avrai una soppressione primaria e secondaria e poi una fase di verifica con tecniche di programmazione lineare per vedere che tutto sia protetto. Il problema di queste cose è la computazionalità del problema.

3.4 Microdati

Facciamo una carrellata sui microdati che sono più vicini ai casi di k -anonymity. Sono nel caso in cui rilascio i microdati e se tu devi fare qualche statistica te la fai tu.

I problemi sono i record linker attach che possono esporre i dati sensibili degli esponenti.

Notiamo che i quasi identificatori sono quelli che permettono di ridurre l'incertezza, non solo che permettono di identificare qualcuno.

Abbiamo tecniche:

- Perturbativa: possono introdurre rumore
- Non perturbativa: diminuiscono il dettaglio dei dati non introducendo rumore

Dobbiamo ad esempio limitare i dettagli geografici, come ad esempio la zona geografica di appartenenza o la temperatura.

Quando ho a che fare con microdati si parlava di protezione andando a modificare i valori con tecniche di prima, oggi, specialmente con il GDPR, molto spesso c'è l'uso dei dati sintetici. Il problema è che con i dati sintetici puoi arrivare fino ad un certo punto, è vero però che puoi avere dei dati quasi-sintetici.

Quando operi con trasformazione vai ad operare sui dati che devono essere rilasciati, se puoi andare a perturbarli dipende dal tipo:

- Continui: stipendio, giorni di ferie, date. Sono dati per cui ha senso fare operazione aritmetiche (se vuoi introdurre rumore ha senso introdurre più o meno)
- Categorici: lavoro, stato civile, razza. Non ha senso fare operazioni aritmetiche

3.4.1 Sampling

Il campionamento, non usi tutti i dati ma ne pubblichi un campione.

3.4.2 Soppressione

Vai a cancellare dei valori, l'abbiamo usata in k-anonymity perchè sopprimevamo una tupla o una cella. Vuol dire andare a sbianchettare. Sopprimono le celle che potrebbero portare a re-identificazione.

3.4.3 Global recoding

É come una generalizzazione, noi usavamo la generalizzazione per proteggere i quasi identifier, possiamo usarla per proteggere anche le operazioni sensibili. Invece di pubblicare i valori precisi ne pubblico una generalizzazione. Sto applicando generalizzazione al valori sensibili. Raggruppo i valori in fasce e poi pubblico la fascia.

SSN	Name	Race	DoB	Sex	ZIP	MarStat	Holidays	Income
		Asian	64/09/27	F	94139	Divorced	13	260
		Asian	64/09/30	F	94139	Divorced	1	170
		Asian	64/04/18	M	94139	Married	40	200
		Asian	64/04/15	M	94139	Married	17	280
		Black	63/03/13	M	94138	Married	2	190
		Black	63/03/18	M	94138	Married	13	185
		Black	64/09/13	F	94141	Married	15	200
		Black	64/09/07	F	94141	Married	60	290
		White	61/05/14	M	94138	Single	17	170
		White	61/05/08	M	94138	Single	10	300
		White	61/09/15	F	94142	Widow	15	200

Global recoding on **Income**:

[150-199]: low, [200-289]: medium, [290-310] high

3.4.4 top-coding o bottom-coding

I valori esposti sono gli outlier, sostanzialmente i valori alla coda della distribuzione. Sostanzialmente mettono delle soglie al di sopra (o sotto) di quel valore quel valore non lo pubblico.

3.4.5 Generalizzazione

É un altro flavour di questa cosa, è esattamente quella che abbiamo visto per k-anonymity, si basa su una gerarchia di generalizzazione

SSN	Name	Race	DoB	Sex	ZIP	MarStat	Holidays	Income
		Asian	64/09/27	F	94139	Divorced	13	260
		Asian	64/09/30	F	94139	Divorced	<10	170
		Asian	64/04/18	M	94139	Married	>30	200
		Asian	64/04/15	M	94139	Married	17	280
		Black	63/03/13	M	94138	Married	<10	190
		Black	63/03/18	M	94138	Married	13	185
		Black	64/09/13	F	94141	Married	15	200
		Black	64/09/07	F	94141	Married	>30	290
		White	61/05/14	M	94138	Single	17	170
		White	61/05/08	M	94138	Single	10	300
		White	61/09/15	F	94142	Widow	15	200

Top-coding on **Holidays** for values higher than 30
Bottom-coding on **Holidays** for values lower than 10

3.4.6 Random noise

Vuol dire perturbare i dati, è quello che ha fatto Netflix quando ha pubblicato ha perturbato i rating dei dati. Quando uso rumore per dati che vanno fuori è utile sapere l'uso dei dati per essere sicuro di non andare a cambiare troppo quello che può essere impattante. Quello che facciamo è preservare la statistica che può essere calcolata sul totale.

3.4.7 Swapping

Faccio lo stesso ragionamento della generalizzazione, è quello che abbiamo visto prima, i dati di uno vanno nell'altro e viceversa. Lo vediamo fatto nella stessa tabella, scambiamo i giorni di vacanza e l'income tenendo fermo il genere e lo stato civile.

SSN	Name	Race	DoB	Sex	ZIP	MarStat	Holidays	Income
		Asian	64/09/27	F	94139	Divorced	13	260
		Asian	64/09/30	F	94139	Divorced	1	170
		Asian	64/04/18	M	94139	Married	40	200
		Asian	64/04/15	M	94139	Married	17	280
		Black	63/03/13	M	94138	Married	2	190
		Black	63/03/18	M	94138	Married	13	185
		Black	64/09/13	F	94141	Married	15	200
		Black	64/09/07	F	94141	Married	60	290
		White	61/05/14	M	94138	Single	17	170
		White	61/05/08	M	94138	Single	10	300
		White	61/09/15	F	94142	Widow	15	200

Scambio rispetto a gente con genere e stato civile uguale:

SSN	Name	Race	DoB	Sex	ZIP	MarStat	Holidays	Income
		Asian	64/09/27	F	94139	Divorced	13	260
		Asian	64/09/30	F	94139	Divorced	1	170
		Asian	64/04/18	M	94139	Married	2	190
		Asian	64/04/15	M	94139	Married	17	280
		Black	63/03/13	M	94138	Married	40	200
		Black	63/03/18	M	94138	Married	13	185
		Black	64/09/13	F	94141	Married	60	290
		Black	64/09/07	F	94141	Married	15	200
		White	61/05/14	M	94138	Single	10	300
		White	61/05/08	M	94138	Single	17	170
		White	61/09/15	F	94142	Widow	15	200

3.4.8 Micro-aggregation

É come fare una computazione a gruppetti, come pubblicare la statistica a gruppetti. Facciamo i gruppi sulla base di stato civile e maritas status:

SSN	Name	Race	DoB	Sex	ZIP	MarStat	Holidays	Income
		Asian	64/09/27	F	94139	Divorced	13	260
		Asian	64/09/30	F	94139	Divorced	1	170
		Asian	64/04/18	M	94139	Married	40	200
		Asian	64/04/15	M	94139	Married	17	280
		Black	63/03/13	M	94138	Married	2	190
		Black	63/03/18	M	94138	Married	13	185
		Black	64/09/13	F	94141	Married	15	200
		Black	64/09/07	F	94141	Married	60	290
		White	61/05/14	M	94138	Single	17	170
		White	61/05/08	M	94138	Single	10	300
		White	61/09/15	F	94142	Widow	15	200

Sostituisco i valori dell'income con i valori che sono la media, per cui per ciascun gruppetto ho già calcolato la media e pubblico quella. Con questo micro blurring l'ultima riga è problematica perchè è espressa, non posso pubblicarla così.

SSN	Name	Race	DoB	Sex	ZIP	MarStat	Holidays	Income
		Asian	64/09/27	F	94139	Divorced	13	215
		Asian	64/09/30	F	94139	Divorced	1	215
		Asian	64/04/18	M	94139	Married	40	213
		Asian	64/04/15	M	94139	Married	17	213
		Black	63/03/13	M	94138	Married	2	213
		Black	63/03/18	M	94138	Married	13	213
		Black	64/09/13	F	94141	Married	15	245
		Black	64/09/07	F	94141	Married	60	245
		White	61/05/14	M	94138	Single	17	235
		White	61/05/08	M	94138	Single	10	235
		White	61/09/15	F	94142	Widow	15	200

3.4.9 Tecniche sintetiche

Introdurre rumore sui piccoli numeri vado a perturbare fortemente i dati, quindi piuttosto preferisco finchè posso avere dati sintetici. Problemi:

- Generare dati sintetici che rispecchino veramente i dati reali
- Dati sintetici che siano abbastanza accurati da riflettere i dati reali ma che non facciano allo stesso tempo leakage dei dati reali

Possiamo usare dati parzialmente sintetici, significa che vado ad aumentare i dati con dati sintetici per andare a coprire certe capacità di identificazione.

Chapter 4

Privacy e protezione dei dati in scenari emergenti

Dobbiamo cercare di capire come proteggere i dati quando affido la loro gestione a servizi esterni. Dobbiamo mettere focus sul mantenere il controllo dei dati che mettiamo fuori, ma accenneremo anche ai problemi di privacy dell'utente.

Non siamo più in un contesto in cui ho i miei dati e li devo pubblicare, ma siamo nel campo in cui abbiamo i dati e vogliamo affidarli a qualcuno per gestirli e qualunque mossa io faccio i dati vengono collezionati, probabilmente in nome di un servizio che qualcuno mi offre.

Viviamo in una società in cui tutto è connesso, tutto è smart in cui le componenti sono connesse. Dal punto di vista della protezione in teoria questa evoluzione dovrebbe essere migliore, perchè abbiamo migliori meccanismi di protezione, migliori garanzie di resistenza rispetto a cose che possono andare male, abbiamo anche una migliore capacità di prevenzione e risposta, anche se c'è qualche anomalia che potrebbe configurare qualche violazione siamo in grado di riconoscerla.

La downside è che viviamo in un sistema iper-complicato in cui perdiamo traccia delle cose, l'anello più debole è quello che può essere soggetto ad hacking, può essere sfruttato per prendere informazioni in modo non autorizzato e sfruttato per casuale danno da chi entra nel sistema.

Il problema di questo sistema è che se tutto è collegato ad internet invece che hackerarti il server di hackerano il frigorifero, però diventa anche un punto di uscita dei dati non solo di ingresso.

La sicurezza in questo caso significa:

- Garantire protezione dell'infrastruttura
- Garantire sicurezza della rete
- Protezione dei device
- Protezione dei dati: quello che ci interessa

I dati sono al centro di quello che è un sistema smart. Avere un sistema intelligente significa avere un sistema che:

- Osserva quello che succede
- Prende dati, condivide estrarli analizzarli
- Estrai conoscenza dai dati

Ci occuperemo sia della confidenzialità sia della sicurezza dei dati e dei processi sia dell'integrità del tutto, avere garanzie di correttezza dei dati. Il problema è che non siamo più in una situazione in cui ho io i dati, ma molto spesso non li teniamo noi, sono memorizzati fuori da qualche parte.

4.1 Cloud Computing

Molto spesso viene usato dai provider di servizi ma ora anche da noi, il fatto di usare un provider esterno per memorizzare dati o per la condivisione o fornire un servizio a qualcuno.

Vantaggi provider esterni:

- Economia di scala
- Dati sempre disponibili
- Infrastruttura che scala in base all'applicazione

Il prezzo che paghi è che perdi diretto controllo dei tuoi dati. Dobbiamo garantire protezione ai dati quando non li abbiamo in mano noi.

Tra i motivi per cui le aziende usano il cloud è anche per avere una maggiore protezione, rispetto ad una protezione sull'intrusione effettivamente il provider fornisce una protezione migliore. Il problema è che il provider garantisce protezione rispetto al mondo esterno, però il provider ha i dati. Ad esempio potremmo criptarli, il problema è che sono criptati per la protezione in storage, però se dobbiamo accedervi dobbiamo fornire al provider anche la chiave, il dato è aperto per chi i dati deve gestirli. Significa che il Cloud deve essere fidato rispetto alla confidenzialità del dato.

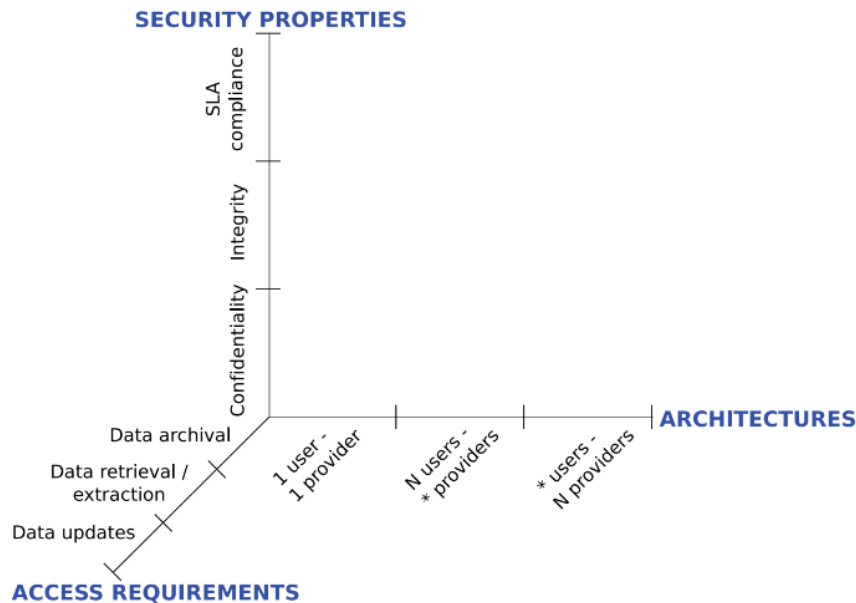
Quello su cui si sta lavorando è avere entrambe le cose, sia avere un server esterno per processare e memorizzare i dati ma senza dare completo accesso alle informazioni al provider che le deve gestire.

Anche concentrandosi solo sulla protezione dei dati non è un solo problema che devo gestire, se i dati li mettiamo fuori abbiamo anche il problema della regolamentazione degli accessi.

4.2 Protezione dati in contesto Cloud

Caratterizziamo la protezione con tre dimensioni:

- Proprietà di sicurezza: paradigma CIA:
 - confidentiality
 - integrity
 - availability
- Requisiti di accesso, cosa ti serve dei dati che metti fuori:
 - Storage
 - Funzionalità accesso: query da fare
 - Dati statici o dinamici
- Architettura:
 - Devo solo io scaricare i miei dati
 - Più utenti: controllo accesso
 - Più provider: condivisione



4.2.1 Security properties

Classico paradigma CIA come detto in precedenza:

- **Confidentiality:** non è solo confidenzialità dei dati, ma è anche rispetto a chi li memorizza. L'altro aspetto è confidenzialità anche su chi accede ai dati (utenti), questo è un problema ortogonale nato prima che ci si occupasse di protezione dati nel cloud. Confidenzialità sulle azioni che l'utente può fare sui dati.
- **Integrità:**
 - Protezione dell'integrità dei dati memorizzati fuori. Viene da un mix tra la capacità di capire se i dati non sono integri e la capacità di fare recovery se i dati non sono corretti, viene fatto con ridondanza
 - Integrità delle risposte alle query, garantire che il risultato della query abbia integrità. Già con tecniche crittografiche riusciamo a vedere che le cose non tornano, per vedere anche completezza dei dati posso legarli e firmarli per vedere se le cose non tornano.
- **Compliance rispetto al SLA:** service level agreement. Se ho pagato per avere tot storage è facile da sapere che viene rispettato. Ma se ho pagato per avere una tot copie di dati o che i dati rimangono in Europa come faccio a controllare?

4.2.2 Access Requirements

- **Data archival:**
 - Usiamo il provider fuori solo per storage, sostanzialmente significa che operiamo a livello di file ma non andiamo mai a livello di granularità più fine
 - Ci dobbiamo preoccupare principalmente della protezione dei dati in storage. Granularità di protezione e di accesso sono allineate.

- Data retrieval/extraction:
 - Abbiamo bisogno di andare a livello più fine, dobbiamo farci delle query. Quello che dobbiamo fare è avere supporto di data processing dal provider esterno
 - Il problema è che se il provider esterno fosse completamente fidato non ne staremmo neanche a parlare, ma se i dati li hai impacchettati per proteggerli hai il problema di come fai a fare le query
- Data update: I dati sono statici o dinamici? metto i dati fuori che poi non cambiano e li leggiamo solo o sono in un contesto in cui i dati fuori devo aggiornarli.
 - Allora comincio ad avere supporto a una funzionalità per modificare i dati che sono protetti
 - Devo proteggere anche le operazioni di scrittura rispetto all'effetto che hanno sui dati

4.2.3 Architettura

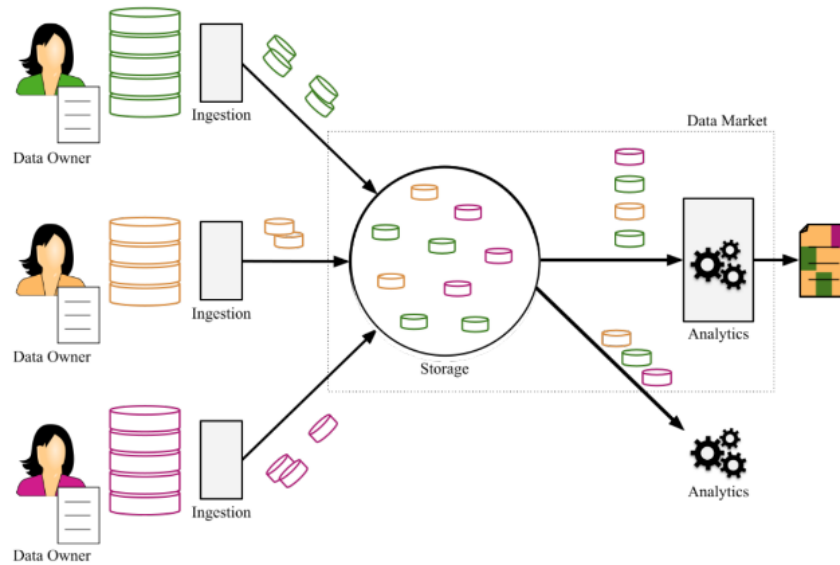
- 1 user - 1 provider: sono in un contesto di one to one:
 - Problema di proteggere dati in storage
 - Problema di dati accesso ai dati in granularità fine
 - Problema di avere garanzie di privacy delle query e di integrità delle query
- n users - 1 provider: potrei dare accessi diversi a diversi utenti:
 - Problema di access control
 - Problema di come gestisco le scritture, devo regolamentare non solo la lettura ma anche la scrittura, dove l'insieme degli scrittori non sarà uguale a quello dei lettori
- n providers: posso avere un contesto in cui dati diversi li metto in provider diversi. Oppure possono essere dati sotto diverse autorità

Lo scenario più semplice possibile è al centro del grafico, quelli più difficili sono ai margini. Ogni combinazione delle differenti istanze delle dimensioni identificano nuovi problemi.

Possiamo essere in un contesto in cui il provider è trustworthy, è il contesto di honest-but-curious in cui ci fidiamo del provider rispetto al servizio ma vogliamo proteggere la confidenzialità anche rispetto al provider.

Ho altri due scenari in cui non ho più un provider trustworthy, non solo devo proteggere la confidenzialità ma devo controllare che sia corretto il comportamento del server:

- Lazy: non spende risorse per fare le cose male
- Malicious: possiamo assumere uno sforzo da qualcuno per fare sì che la violazione avvenga. Il provider può subire delle violazioni.



4.3 Digital Data Market

Hai sostanzialmente uno spazio comune a cui diverse parti contribuiscono per avere una analisi comune per sfruttare dati insieme.

Bisogna essere in grado di attaccare una politica al dato che poi lo segue per tutto il ciclo di vita. Problemi e tecniche di protezione dei dati:

- Sanitization: faccio offuscation e non rilascio il dato nella sua interezza
- Wrapping: impacchetto il dato per una sua protezione stile encryption

Sanitization non è reversibile, Wrapping è impacchettare i dati con un livello di protezione tipo encryption che puoi usare come storage communication, potresti farla anche runtime che però è reversibile lungo tutto il ciclo di vita del dato.

4.4 Problemi da guardare

- Privacy per l'utente
- Data protection
- Query execution
- Private access
- Data integrity and correctness
- Access control enforcement
- Data publication and utility
- Collaborative query execution

Chapter 5

Preferenze di privacy degli utenti

Il problema di far rimanere confidenziale l'identità degli utenti in un sistema aperto può essere aggiunta a non dover comunque mantenere tutte le informazioni dell'utente lato server. Questo problema è partito ancora prima del cloud, la cosa era mantenere confidenzialità dell'identità degli utenti.

- Tecniche di comunicazione anonime: tor. Guarda l'anonimità del ricevente e mittente nel contesto di comunicazione
- Privacy degli utenti in servizi basati sulla posizione
- Controllo dell'accesso basato sugli attributi, le autorizzazioni dipendono dalle proprietà che hai
- Supporto per le preferenze di privacy dell'utente

5.1 User empowerment

Ragioniamo su dei metodi per dare controllo all'utente sul rilascio delle proprie informazioni in un contesto di comunicazione con il provider. Hai bisogno di regolamentare le tue informazioni per:

- Scopo primario: dai la carta di credito a scopo pagamento. È il rilascio diretto.
- Uso secondario: una volta che hai rilasciato le informazioni cosa può fare l'altra parte con le tue informazioni. Se deve cancellarla, memorizzarle in forma criptata, darne notifica a qualcuno.

Quello che guardiamo è il problema di rilascio delle informazioni in contesto di interazioni e a regolamentare l'uso primario.

Questo va in un contesto di cambiamento di filosofia riguardo le politiche di accesso, eravamo abituati ad avere diversi servizi di sicurezza per cui avevamo l'utente, la prima cosa che è l'autenticazione dell'utente, poi il controllo dell'accesso, poi un servizio di audit e l'encryption, se la usi, la usi per protezione dello storage autenticazione delle parti. Questa divisione classica è andata a sfumarsi, ad esempio la separazione tra authentication ed access control è andata un po' a sparire, c'è un controllo dell'accesso basato sulle caratteristiche degli attributi che vuole leggere l'utente. Questo approccio è chiamato attribute-based access control, voglio leggere il file ma non mi risponde sì o no, devo presentare un certificato che mi dice se posso accedervi o no. Questa piccola osservazione è un grosso cambiamento dell'engine che deve fare il controllo degli accessi.

Devo avere diverse politiche:

- Come specifico le regole di autorizzazione
- Come le valuto? non è più un mondo chiuso
- Dobbiamo comunicare, devi fidarti per darmi il tuo token. Abbiamo il problema di come comunicare dall'altra parte

- Strategie di trust management: viene fuori il dialogo tra utente e server. Il server chiede qualcosa per avere le credenziali allora noi gli chiediamo qualcosa per sapere se è fidato e così via.
- Garanzie di terminazione del dialogo utente-server
- Garanzie di correttezza del dialogo

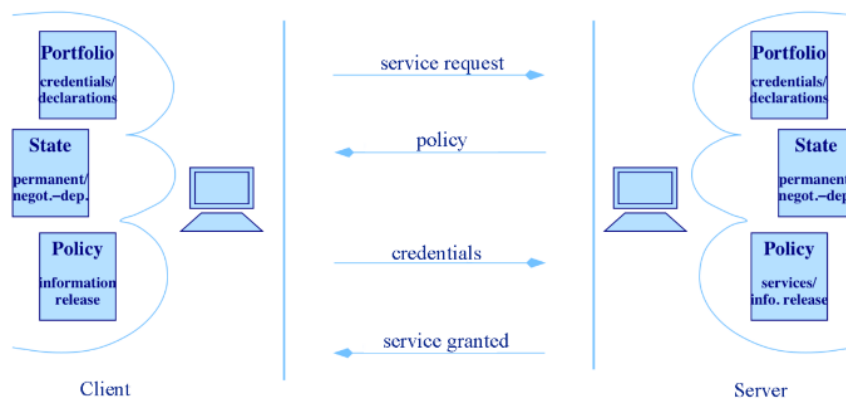
Interactive access control

Client e server hanno:

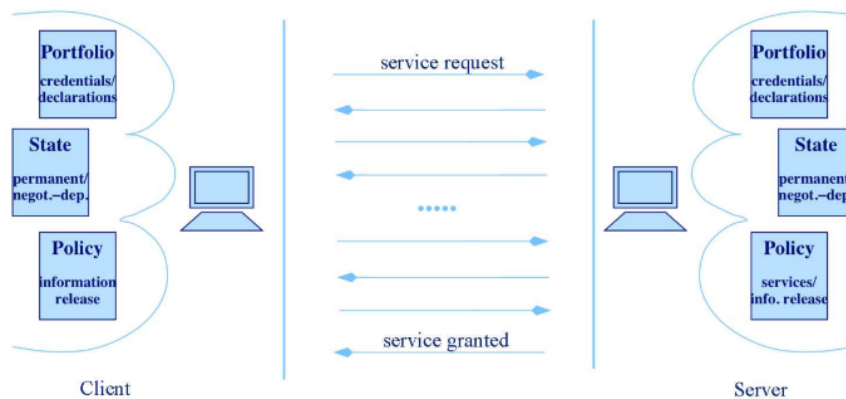
- Portfolio di certificati: lato client sono i certificati che deve dare lato server sono i certificati che dimostrano che è un server certificato
- politica di rilascio
- Uno stato per cui prendere delle decisioni

Dialogo classico:

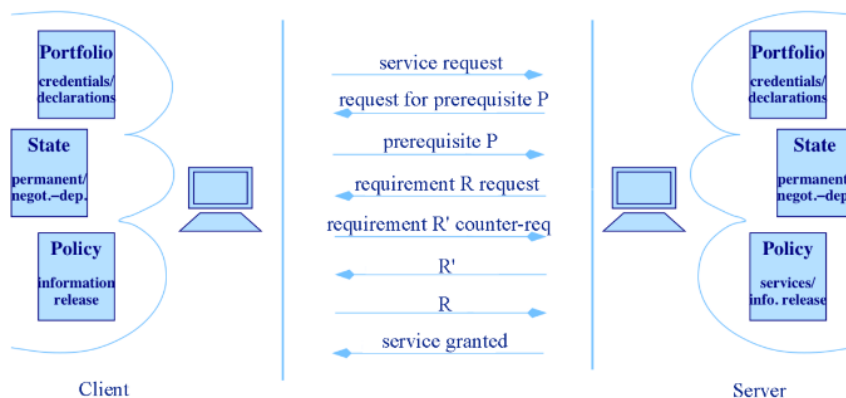
- Chiedo l'accesso: voglio vedere il film
- Ti dice la politica: hai pagato?
- Gli fai vedere le credenziali
- Hai l'accesso



Si è visto che è possibile avere un'interazione migliore, ti chiedo tante cose, è una multi-step negotiation:



Una cosa che è una via di mezzo è distinguere quelle che sono le informazioni necessarie per l'accesso e quelle sufficienti, in modo da essere anche bravi rispetto all'utente che distingue quello che ho bisogno di sapere per darti garanzie di accesso, sostanzialmente gli dico che cosa deve fornirmi per avere l'accesso, così può decidere se dare le informazioni o meno. Si chiama two-step interaction e serve per separare quelli che sono prerequisiti e quelli che sono requisiti per l'accesso:



Esistono dei nuovi certificati che permettono di presentare proprietà a livello di granularità più fine, o addirittura posso dimostrare la veridicità delle mie proprietà senza rilasciare il certificato. Possono anche dimostrare che ho il certificato senza rilasciarlo. Dall'altra parte approccio per il controllo dell'accesso che riescono a ragionare sui certificati.

Le cose positive dell'user privacy access:

- È espressivo e potente
- Permette all'utente di specificare quali tipi di informazioni vuole rilasciare

Il lato negativo è che se vogliamo fare esprimere le preferenze di privacy dell'utente ragionare con una classica politica dell'accesso lato utente non è propriamente adeguato, perchè l'utente non ragiona così. Se ci mettiamo nei panni dell'utente che, data una politica, deve fornire un documento, il tipo di cose che l'utente deve poter scegliere:

- Restrizione sulla base del contenuto: se sto comprando ti do la carta di credito, se sto solo navigando il servizio no
- Non voglio che certe informazioni vadano insieme
- Puoi voler controllare l'ammontare di informazioni che rilasci dall'altra parte
- Vuoi limitare l'informazione che hai
- Restrizioni basate sull'istanza: se la carta di credito sta per scadere te la do
- Restrizioni basate sulla storia: se ti ho sempre dato la CI ti continuo a dare la CI
- Rilasciare la prova evitando di dare informazioni

5.2 User privacy preferences: Some approaches

Vediamo diversi approcci ai problemi:

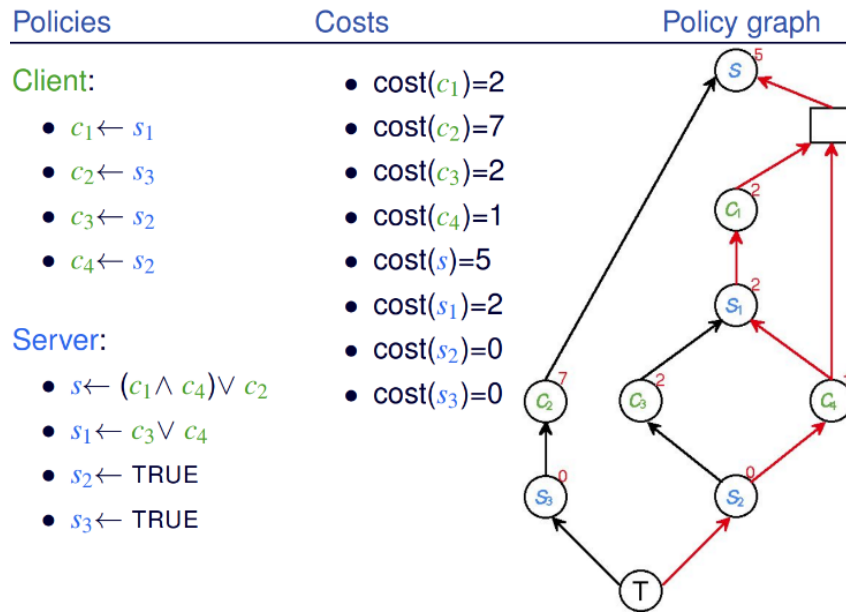
- Cost-sensitive trust negotiation
- Point-based trust management
- Logic-based minimal credential disclosure
- Privacy preferences in credential-based interactions

5.2.1 Cost-sensitive trust negotiation

Siamo ancora in un contesto in cui qualcuno ha un servizio (server) e qualcuno vuole il servizio. Il server per rilasciare il servizio vuole certe proprietà e il client vuole delle garanzie per rilasciare quelle proprietà. Con diverse proprietà che hanno diversi valori per l'utente, ci sono proprietà più o meno sensibili (potremmo valutare il passaporto più sensibile del tesserino universitario).

Il client ha una politica che rilascia se tu hai, ad esempio rilascio 1 se hai s_1 e così via. Il server dice che rilascia s se ha c_1 e c_4 oppure c_2 e così via. Se vogliamo queste politiche le possiamo combinare in un grafo in cui ci dice che se vogliamo ottenere un certo servizio viene erogato solo se hai quello che ci sono nei nodi con archi entranti in S :

Significa che per arrivare ad un servizio devi avere un cammino che ti ci porta. Dicono poi che non tutte le credenziali hanno la stessa sensibilità, quindi il costo della credenziale è la sensibilità ovvero il valore che la parte che ha in mano la credenziale dà a quella credenziale, danno un costo sia client che server. La cosa complicata è che dovremmo avere una comunicazione che porta alla minimizzazione dei costi però nessuno vede il grafo completo. Non valutano il fatto che si abbiamo le credenziali sensibili e mettono client e server sullo stesso piano.



5.2.2 Point-based trust management model

Il Server associa ad ogni credenziale un valore, per fare qualcosa devi superare un certo livello di threshold. Dall'altro lato abbiamo il client che ha le credenziali che considera sensibili che deve fornire per avere l'accesso.

Abbiamo visto come avere la patente a New York:

Threshold of accessing a resource: 10

SERVER

	College ID	Driver's license	Credit card	SSN
Point value	3	6	8	10

CLIENT

	College ID	Driver's license	Credit card	SSN
Sensitivity score	10	30	50	100

Client's options:

- SSN [Points: 10; Sensitivity: 100]
- College ID, Credit card [Points: 11; Sensitivity: 60]
- Driver's license, Credit card [Points: 14; Sensitivity: 80]

Rilasciamo la combinazione di credenziali che ci serve per passare la threshold ma minimizzando i punti di sensibilità che avremmo nel rilascio .

Lo complicano un pochino perchè non danno per conosciuta la threshold, è solo una complicazione teorica perchè nella realtà non succede mai.

5.2.3 Logic-based trust management

Il problema in questo caso è minimizzare il rilascio di informazioni. Anche qui parte da un contesto di negoziazione ma non si pone come faccio la negoziazione, hai un client ed un server, delle credenziali semplificate con il server e il client possono chiedere qualcosa. Il problema che ci si pone è come fa l'utente a specificare una politica di preferenza.

Abbiamo la politica dell'utente (rilascia delle proprietà solo se mi da quello che c'è a destra della freccia) e la politica del server

Alice's policy	On-line book shop's policy	Negotiation paths											
$c_{name} \leftarrow \text{TRUE}$ $c_{bdate} \leftarrow c_{bbb}$ $c_{telephone} \leftarrow c_{bbb}$ $c_{email} \leftarrow c_{bbb}$ $c_{pcode} \leftarrow c_{bbb}$ $c_{id} \leftarrow c_{bbb}$ $c_{passport} \leftarrow c_{bbb}$ $c_{bname} \leftarrow c_{bbb} \wedge c_{osc}$ $c_{baccount} \leftarrow c_{bbb} \wedge c_{osc}$ $c_{credit_card} \leftarrow c_{bbb} \wedge c_{osc}$ $c_{pin} \leftarrow c_{bbb} \wedge c_{osc}$	$\text{purchase} \leftarrow p_{register} \wedge p_{payment}$ $p_{register} \leftarrow (c_{name} \wedge c_{bdate} \wedge (c_{email} \vee c_{pcode})) \vee c_{id} \vee c_{passport} \vee ((c_{name} \vee c_{email}) \wedge c_{id})$ $p_{payment} \leftarrow (c_{bname} \wedge c_{baccount}) \vee (c_{credit_card} \wedge c_{pin})$ $c_{bbb} \leftarrow \text{TRUE}$ $c_{osc} \leftarrow \text{TRUE}$												
		name	bdate	telephone	email	pcode	id	passport	bname	baccount	credit_card	pin	
		S_1	x	x	x					x	x		
		S_2	x	x	x							x	
		S_3	x	x		x			x	x			
		S_4	x	x		x					x	x	
		S_5						x	x	x			
		S_6						x			x	x	
		S_7							x	x	x		
		S_8							x		x	x	
		S_9	x					x	x	x			
		S_{10}	x					x			x	x	
		S_{11}			x	x			x	x			
		S_{12}			x	x					x	x	

Volendo potremmo mettere questa cosa in un grafo come facevamo prima. Se vediamo il server da tutto, non pone alcuna condizione, quindi ci mettiamo lato client. Nella tabella vediamo tutte le possibili strategie per potersi registrare e pagare. Ciascuna strategia posso vederla come un vettore dove 0 non rilascio e 1 rilascio, a questo punto che strategia è meglio di un'altra? non lo chiediamo a run time ma chiedendo di specificare una politica.

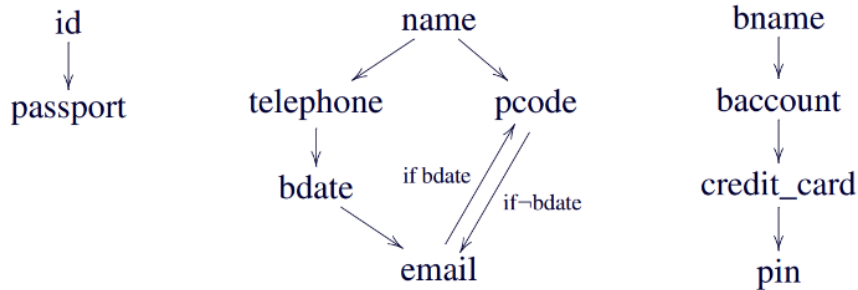
Alice's policy	On-line book shop's policy	Negotiation paths																																																																																																																																																												
$c_{name} \leftarrow \text{TRUE}$ $c_{bdate} \leftarrow c_{bbb}$ $c_{telephone} \leftarrow c_{bbb}$ $c_{email} \leftarrow c_{bbb}$ $c_{pcode} \leftarrow c_{bbb}$ $c_{id} \leftarrow c_{bbb}$ $c_{passport} \leftarrow c_{bbb}$ $c_{bname} \leftarrow c_{bbb} \wedge c_{osc}$ $c_{baccount} \leftarrow c_{bbb} \wedge c_{osc}$ $c_{credit_card} \leftarrow c_{bbb} \wedge c_{osc}$ $c_{pin} \leftarrow c_{bbb} \wedge c_{osc}$	$\text{purchase} \leftarrow p_{register} \wedge p_{payment}$ $p_{register} \leftarrow (c_{name} \wedge c_{bdate} \wedge (c_{email} \vee c_{pcode})) \vee c_{id} \vee c_{passport} \vee ((c_{name} \vee c_{email}) \wedge c_{id})$ $p_{payment} \leftarrow (c_{bname} \wedge c_{baccount}) \vee (c_{credit_card} \wedge c_{pin})$ $c_{bbb} \leftarrow \text{TRUE}$ $c_{osc} \leftarrow \text{TRUE}$	<table><tr><th></th><th>name</th><th>bdate</th><th>telephone</th><th>email</th><th>pcode</th><th>id</th><th>passport</th><th>bname</th><th>baccount</th><th>credit_card</th><th>pin</th></tr><tr><td>S_1</td><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>S_2</td><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td></tr><tr><td>S_3</td><td>1</td><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>S_4</td><td>1</td><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td></tr><tr><td>S_5</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>S_6</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td></tr><tr><td>S_7</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>S_8</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td><td>1</td></tr><tr><td>S_9</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>S_{10}</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td></tr><tr><td>S_{11}</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>S_{12}</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td></tr></table>		name	bdate	telephone	email	pcode	id	passport	bname	baccount	credit_card	pin	S_1	1	1	0	1	0	0	0	1	1	0	0	S_2	1	1	0	1	0	0	0	0	0	1	1	S_3	1	1	0	0	1	0	0	1	1	0	0	S_4	1	1	0	0	1	0	0	0	0	1	1	S_5	0	0	0	0	0	1	0	1	1	0	0	S_6	0	0	0	0	0	1	0	0	0	1	1	S_7	0	0	0	0	0	0	1	1	1	0	0	S_8	0	0	0	0	0	0	1	0	0	1	1	S_9	1	0	0	0	0	1	0	1	1	0	0	S_{10}	1	0	0	0	0	1	0	0	0	1	1	S_{11}	0	0	0	1	0	1	0	1	1	0	0	S_{12}	0	0	0	1	0	1	0	0	0	1	1
	name	bdate	telephone	email	pcode	id	passport	bname	baccount	credit_card	pin																																																																																																																																																			
S_1	1	1	0	1	0	0	0	1	1	0	0																																																																																																																																																			
S_2	1	1	0	1	0	0	0	0	0	1	1																																																																																																																																																			
S_3	1	1	0	0	1	0	0	1	1	0	0																																																																																																																																																			
S_4	1	1	0	0	1	0	0	0	0	1	1																																																																																																																																																			
S_5	0	0	0	0	0	1	0	1	1	0	0																																																																																																																																																			
S_6	0	0	0	0	0	1	0	0	0	1	1																																																																																																																																																			
S_7	0	0	0	0	0	0	1	1	1	0	0																																																																																																																																																			
S_8	0	0	0	0	0	0	1	0	0	1	1																																																																																																																																																			
S_9	1	0	0	0	0	1	0	1	1	0	0																																																																																																																																																			
S_{10}	1	0	0	0	0	1	0	0	0	1	1																																																																																																																																																			
S_{11}	0	0	0	1	0	1	0	1	1	0	0																																																																																																																																																			
S_{12}	0	0	0	1	0	1	0	0	0	1	1																																																																																																																																																			

Disclosure sets are represented as binary vectors
 $\Rightarrow$ 0 means do not disclose; 1 means disclose

Nonostante la politica possiamo vedere che alcune strategie sono brutte, ci sono strategie che non minimizzano le credenziali da fornire, ad esempio la scelta con tutti 1 da il

servizio ma non ha senso dare tutto. Una regola generale è che 0 è migliore di 1. Quindi noi possiamo scegliere quali scelte dominano altre, ad esempio sappiamo che S_5 domina S_9 , ma per quelle pari deve essere l'utente che sceglie cosa darci.

In questo caso l'utente non ha un costo per ogni credenziale ma da una classifica di preferenze:



Quindi in un contesto di questo tipo significa che inizialmente rimuoviamo tutte le scelte che vengono dominate da altre:

Disclosure sets

	name	bdate	telephone	email	pcode	id	passport	bname	baccount	credit_card	pin
S_1	1	1	0	1	0	0	0	1	1	0	0
S_2	1	1	0	1	0	0	0	0	0	1	1
S_3	1	1	0	0	1	0	0	1	1	0	0
S_4	1	1	0	0	1	0	0	0	0	1	1
S_5	0	0	0	0	0	1	0	1	1	0	0
S_6	0	0	0	0	0	1	0	0	0	1	1
S_7	0	0	0	0	0	0	1	1	1	0	0
S_8	0	0	0	0	0	0	1	0	0	1	1
S_9	1	0	0	0	0	1	0	1	1	0	0
S_{10}	1	0	0	0	0	1	0	0	0	1	1
S_{11}	0	0	0	1	0	1	0	1	1	0	0
S_{12}	0	0	0	1	0	1	0	0	0	1	1

Pareto composition

S_5 dominates S_9 since $0 \succ_{name} 1$
 S_5 dominates S_{11} since $0 \succ_{email} 1$
 S_6 dominates S_{10} since $0 \succ_{name} 1$
 S_6 dominates S_{12} since $0 \succ_{email} 1$

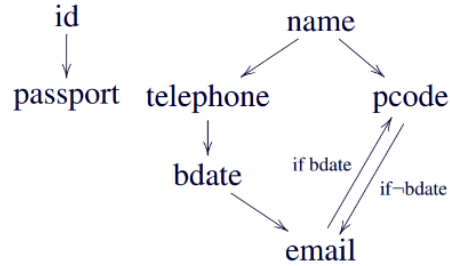
poi andiamo a vedere cosa preferisce l'utente così trovo le altre scelte dominanti:

Disclosure sets

	name	bdate	telephone	email	pcode	id	passport	bname	baccount	credit_card	pin
S_1	1	1	0	1	0	0	0	1	1	0	0
S_2	1	1	0	1	0	0	0	0	0	1	1
S_3	1	1	0	0	1	0	0	1	1	0	0
S_4	1	1	0	0	1	0	0	0	0	1	1
S_5	0	0	0	0	0	1	0	1	1	0	0
S_6	0	0	0	0	0	1	0	0	0	1	1
S_7	0	0	0	0	0	0	1	1	1	0	0
S_8	0	0	0	0	0	0	1	0	0	1	1
S_9	1	0	0	0	0	1	0	1	1	0	0
S_{10}	1	0	0	0	0	1	0	0	0	1	1
S_{11}	0	0	0	1	0	1	0	1	1	0	0
S_{12}	0	0	0	1	0	1	0	0	0	1	1

Hierarchical preferences

S_5 dominates S_7
 S_6 dominates S_8
 S_1 dominates S_3
 S_2 dominates S_4

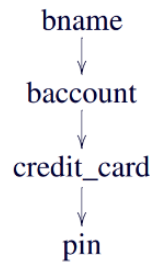


Disclosure sets

	name	bdate	telephone	email	pcode	id	passport	bname	baccount	credit_card	pin
S_1	1	1	0	1	0	0	0	1	1	0	0
S_2	1	1	0	1	0	0	0	0	0	1	1
S_3	1	1	0	0	1	0	0	1	1	0	0
S_4	1	1	0	0	1	0	0	0	0	1	1
S_5	0	0	0	0	0	1	0	1	1	0	0
S_6	0	0	0	0	0	1	0	0	0	1	1
S_7	0	0	0	0	0	0	1	1	1	0	0
S_8	0	0	0	0	0	0	1	0	0	1	1
S_9	1	0	0	0	0	1	0	1	1	0	0
S_{10}	1	0	0	0	0	1	0	0	0	1	1
S_{11}	0	0	0	1	0	1	0	1	1	0	0
S_{12}	0	0	0	1	0	1	0	0	0	1	1

Transitive combination of preferences

S_1 dominates S_2
 S_5 dominates S_6



⇒ user has to choose between S_1 , S_5

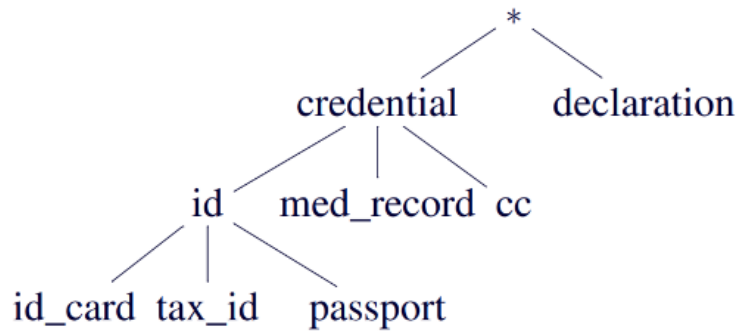
Non è detto che sia meglio specificare preferenze che valori, è più vicino al modo di ragionare dell'utente ma ancora in modo semplificato perchè abbiamo una credenziale un attributo e regole semplici.

5.2.4 Privacy Preferences in Credential-based Interactions

Cerchiamo di catturare le credenziali e come supportare l'utente nella specifica di cosa vuole rilasciare o meno quindi entrano in gioco le preferenze dell'utente.

Stiamo parlando in un contesto di interazione client-server, abbiamo questo concetto di credenziali che ci certificano le proprietà. Ogni credenziale ha un tipo un identificatore e qualcuno che ha firmato quella credenziale. La dichiarazione è self-signed credential.

Abbiamo una gerarchia di credenziali e proprietà:



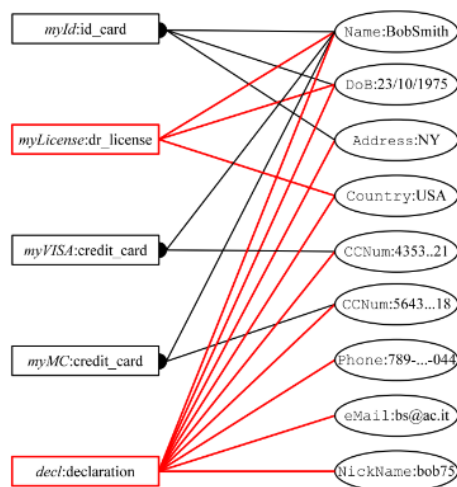
Vediamo i tipo di proprietà e di credenziali, ci sono proprietà che sono le stesse indipendentemente dal certificato (ad esempio nome e cognome) sono quindi proprietà indipendenti dalle credenziali. Poi ci sono le proprietà che dipendono dalla credenziali (ad esempio il numero del passaporto).

Sulle credenziali facciamo una distinzione tra:

- Credenziali atomiche: quando rilascio rilascio tutte le sue proprietà
- Non atomica: posso estrarre da questa una o più informazioni selettivamente e rilasciare quello che scelgo

Atomic: released as a whole (e.g., X.509)

Non-atomic: properties can be selectively released, proof-of-possession can be certified (e.g., Idemix, U-Prove)

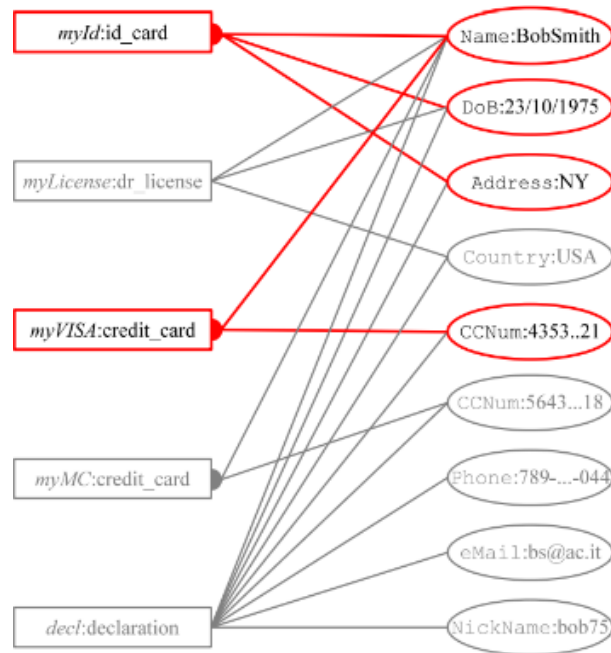


5.2.5 Disclose

Un rilascio è un sottoinsieme delle mie proprietà che deve soddisfare due cose:

- Ogni proprietà deve essere certificata
- Atomicità: se una proprietà di una credenziale atomica è divulgata, tutte le sue proprietà sono divulgate

Questo a rappresentare cos'è un rilascio.



5.2.6 Sensibilità portfolio

Prima avevamo due approcci:

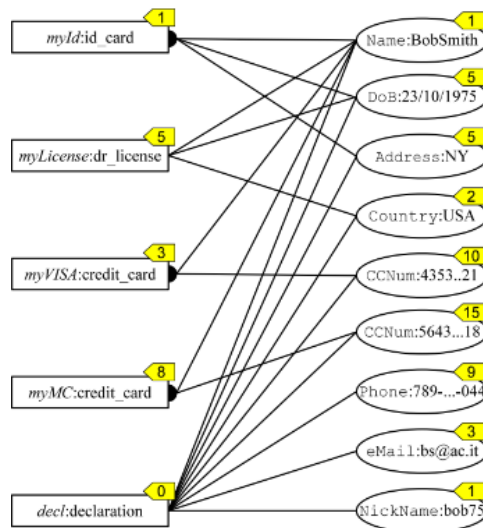
- Un numero
- Preferenze

Il problema delle preferenze è che lasciano incertezza ed è difficile che rilascino una politica completa (se non ho dominanza mi arrangio). D'altro canto specificare dei valori è più semplice quando gioco nel piccolo, quando gioco nel grande è comunque un casino. Non è chiaro cosa voglia dire un numero, è per questo che non è facile traslare una politica di numeri a preferenze e viceversa.

Quindi la sensibilità posso specificarla:

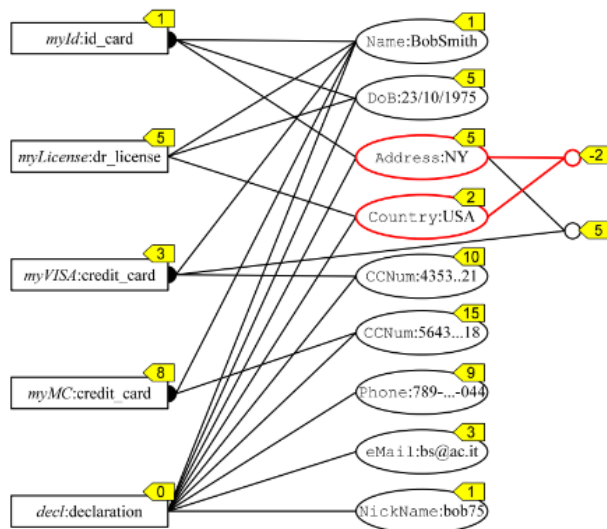
- Gerarchia di relazioni
- Composizione arbitraria di valori

Dando una gerarchia alla sensibilità delle proprietà possiamo sapere cosa all'utente è più caro:

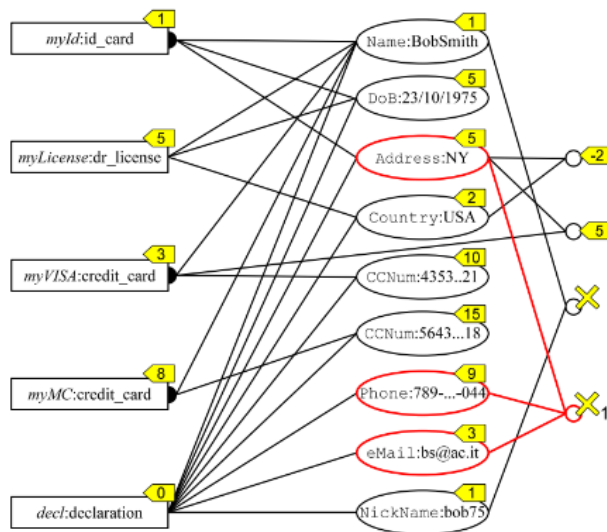


In questo caso abbiamo anche una sensibilità associata alla credenziale perchè il fatto di possedere una credenziale è di per sè sensibile.

Possiamo anche spingerci un po' avanti perchè sappiamo che due pezzi di informazione possono dire molto di più rispetto all'unione dei due pezzi. Possiamo anche esprimere la sensibilità dell'associazione. Attenzione che potrebbe essere anche meno la sensibilità se rilasciamo più attributi, ad esempio se sai che sono a New York sai che sono negli stati uniti quindi l'unione delle due informazioni non è la somma delle sensibilità perchè uno implica l'altro:

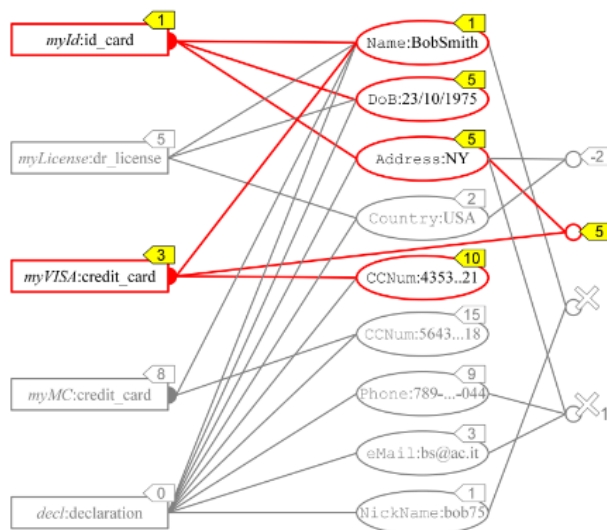


Posso pensare di poter esprimere delle limitazioni sui rilasci. Supponiamo di avere il nome e avergli dato un valore e un nickname a cui ho dato un valore, voglio che quelle due cose non vengano mai viste insieme. Altra cosa può essere una limitazione sul rilascio, tra questi al massimo ne esponi uno:



A questo punto ci chiediamo se il rilascio è valido, dal punto di vista di validità considerando quello che l'utente ci ha chiesto dobbiamo essere sicuri che i constraint dati dall'utente siano soddisfatti.

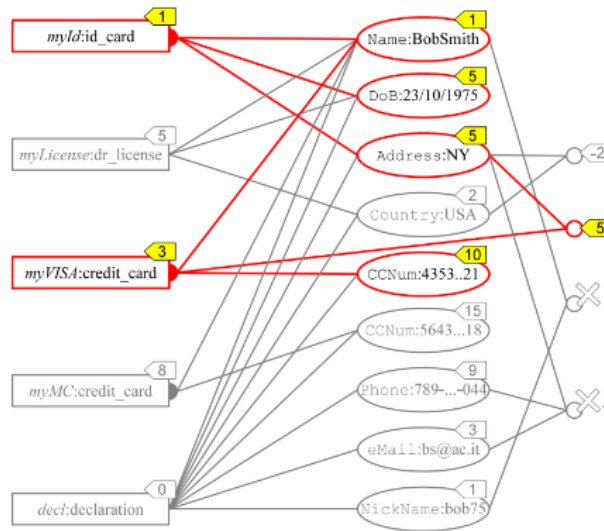
Quindi per calcolare la sensibilità, dato che è espressa in numeri, sommiamo la sensibilità delle credenziali, con attenzione anche le unioni sensibili e le proprietà:



Questo lato client, posso rilasciare le mie proprietà ed ogni rilascio ha una sensibilità. In un sistema vero prima di questa roba ho il problema di come dialogo con il server. Il sever si esprimerà in termini di richieste "dammi certe proprietà con delle credenziali".

Un rilascio soddisfa una richiesta del server se il certificato rilasciato soddisfa il tipo della richiesta. Ad esempio se mi chiede il nome e il numero di un documento posso rilasciare la CI che però contiene anche la data di nascita ad esempio.

$$\mathcal{R} = id.\{Name, Address\} \wedge cc.\{Name, CCNum\}$$



$$\lambda(\mathcal{D}') = 30 \Rightarrow \mathcal{D}' \text{ is minimum}$$

Ho vari problemi:

- Non so più con che server sto parlando
- Problema di dialogo

5.2.7 Server-side issues

Una nota lato server, gli manca la capacità di ragionamento e di dialogo, ad esempio il fatto di dire cose del tipo "voglio il tuo nome certificato da un certificato di un certo tipo", non ha supporto per astrazioni (un documento può essere di un certo tipo piuttosto che di un altro). Non ha supporto al dialogo della politica con l'utente, nel senso che ci potrebbero essere scelte di rilascio più protettivi per la politica di rilascio ma meno per l'utente e viceversa.

Inoltre non ha il supporto per il ragionamento ricorsivo

Chapter 6

Protezione dei dati in outsourcing

Vogliamo avere una granularità per non dover portarmi a casa tutti i dati ma avere computazione lato server. Vogliamo sia confidenzialità sia supporto esecuzione di query stando attento che non mi esponga dati sensibili. Toccheremo anche nel caso in cui abbiamo più utenti coinvolti avendo il controllo dell'accesso, devo dare accesso agli utenti non solo fare le query.

Le soluzioni possono essere basate su:

- encryption
- encryption e fragmentation
- fragmentation

I casi senza encryption sono quelli in cui quello che devi mantenere sensibile non sono i singoli valori ma sono le associazioni dei valori che hai.

6.1 Encryption

Quando vado ad operare in un contesto gestione dati con encryption ho diverse gradazioni di soluzione di protezioni, quelle con garanzie migliori di protezioni e quelle con maggiori funzionalità ma minori garanzie. Chi mi ospita i dati e chi mi dovrebbe fornire i dati potrebbe essere un server onesto ma curioso, nel senso che è fidato ma noi vogliamo proteggere la confidenzialità anche rispetto al server. La prima cosa che ci viene in mente è che se i dati sono criptati possiamo wrappare i dati in un layer di encryption, chi ha la chiave può aprire questo layer ed accedere ai dati. Se stiamo in un contesto relazionale abbiamo delle tabelle in cui abbiamo righe e colonne, l'encryption potremmo averla a livelli diversi criptare la tabella intera, ciascuna riga, ciascuna cella. Chiaro che se cryptiamo a livello di file la granularità della tabella rimane quella, dobbiamo portarci a casa tutta la tabella per fare delle query. Tipicamente le soluzioni dei DB operano a livello di tupla.

Fuori abbiamo un cloud provider in cui memorizziamo una tabella criptata e la chiave se l'abbiamo noi proteggiamo i dati anche a livello del provider.

Facciamo una carrellata veloce.

6.1.1 Keyword-based searches

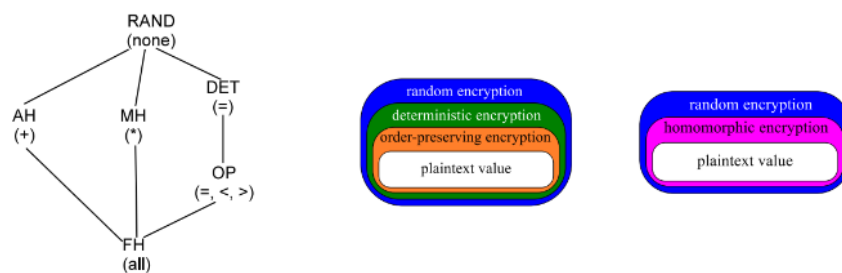
Abbiamo tutto un filesystem criptato e vogliamo trovare tutti i file con dentro qualcosa, quindi il fatto di essere in grado di fare keyword search su stringhe in un testo criptato.

6.1.2 Homomorphic encryption

Homomorphic encryption significa avere una crittografia che supporta direttamente le operazioni sui dati, significa che se devi fare un operazione o cercare qualcosa invece che decriptare ed eseguire l'operazione puoi fare l'operazione direttamente sul criptato.

6.1.3 Onion encryption

Onion encryption: se guardi la crittografia che hai fuori sapendo che ci sono encryption diverse, qualsiasi funzionalità che rispetto nell'encryption è un possibile canale di inferenza. Potremmo usare una crittografia completamente casuale che protegge i dati ma non da funzionalità, potremmo usare una full homomorphic encryption, un homomorphic additivo o moltiplicativo, una deterministic encryption, un order preserving encryption (ordine del chiaro anche nel criptato). Ciascuno di questi viene con un supporto di confidenzialità, l'idea è che vari strati garantiscono query diverse quindi quando arriva una query posso "pelare" la cipolla per fare la query che mi serve

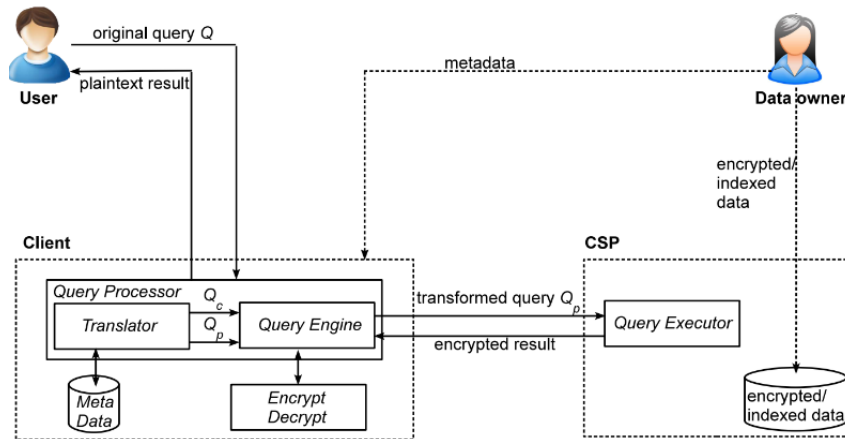


6.1.4 Indexes

L'alternativa è dimenticarsi di querare l'encryption, ma associare ai dati degli indici. Significa avere una tabella con le tue tuple dei metadati e quello che vai a querare è il metadato associato alle tuple. Cripti le tuple e poi per ciascun attributi hai un indice che ti supporta delle condizioni, il problema è come fare a specificare quegli indici, si fanno gli stessi discorsi per l'encryption.



Lo scenario che abbiamo è questo, abbiamo i dati fuori che eventualmente sono indicizzati, quello del service provider è un ambiente non trusted, data una query passiamo in un ambiente trusted che traduce la query sul DB in chiaro in una query che deve operare sul DB criptato:



Cosa sono questi indici? noi vogliamo un indice per ogni attributo sul quale posso eseguire delle condizioni.

Direct (1:1)

Quello più semplice che possiamo avere e anche quello più preciso per equality queries è usare il valore (non criptiamo) o qualche codifica. Abbiamo una corrispondenza di plain text rispetto all'indice diretta, se il valore non è confidenziale potremmo usare semplicemente quel valore lì. Il problema è che è soggetto ad attacchi di frequenza, ad esempio se so che Angel ha due conti e tutti gli altri uno guardando l'encryption riesco a spottarlo:

Patients				Patients ^k					
SSN	Name	Illness	Doctor	Tid	Etuple	I _S	I _N	I _I	I _D
123...89	Alice	Asthma	Angel	1	x4Z3tfX2ShOSM	π	κ	α	δ
234...91	Bob	Asthma	Angel	2	mNHg1oC010p8w	ϖ	ω	α	δ
345...12	Carol	Asthma	Bell	3	WslaCvfyF1Dxw	ξ	λ	α	ν
456...23	David	Bronchitis	Clark	4	JpO8eLTVgwV1E	ρ	ν	β	γ
567...34	Eva	Gastritis	Dan	5	qctG6XnFNDTQc	ι	μ	α	σ
232...11	Eva	Stroke	Ellis	6	kotG8XnFNDTaW	χ	σ	β	ψ

Bucket (n:1)

Posso usare un indicizzazione con collisione se voglio proteggermi da attacchi di frequenza, posso avere diversi valori che si mappano sullo stesso indice. In realtà sappiamo che asma sta solo in alpha. Un problema è che se vogliamo tutti gli asma dobbiamo portarci a casa tutti gli alpha e ci portiamo a casa tuple spurie dobbiamo quindi fare un post processing lato client per rimuovere le tuple spurie. Riesce a supportare equality queries.

Patients			
SSN	Name	Illness	Doctor
123...89	Alice	Asthma	Angel
234...91	Bob	Asthma	Angel
345...12	Carol	Asthma	Bell
456...23	David	Bronchitis	Clark
567...34	Eva	Gastritis	Dan
232...11	Eva	Stroke	Ellis

Patients ^k					
Tid	Etuple	I _S	I _N	I _I	I _D
1	x4Z3tfX2ShOSM	π	κ	α	δ
2	mNHg1oC010p8w	ϖ	ω	α	δ
3	WslaCvfyF1Dxw	ξ	λ	α	ν
4	JpO8eLTVgwV1E	ρ	ν	β	γ
5	qctG6XnFNDTQc	ι	μ	α	σ
6	kotG8XnFNDTaW	χ	$\omicron$	β	ψ

Flattened (1:n)

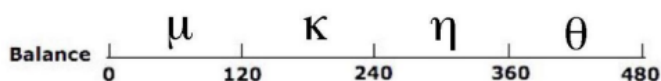
Un'altra idea è usare indici flat, vado ad appiattare nel senso che ciascun indice appare lo stesso numero di volte, la cosa più semplice è che ogni tupla viene rappresentata con indici diversi, il problema di questa cosa è che valori uguali vengono rappresentati con indici diversi. Questo significa che rispetto ad osservazioni statiche non si capisce niente però il problema è come facciamo le query? dovremmo chiedere tutti i possibili indici in cui può essere stato mappato un elemento questo lo rende esposto ad osservazioni dinamiche perchè i valori uguali sono sempre considerati insieme

Patients			
SSN	Name	Illness	Doctor
123...89	Alice	Asthma	Angel
234...91	Bob	Asthma	Angel
345...12	Carol	Asthma	Bell
456...23	David	Bronchitis	Clark
567...34	Eva	Gastritis	Dan
232...11	Eva	Stroke	Ellis

Patients ^k					
Tid	Etuple	I _S	I _N	I _I	I _D
1	x4Z3tfX2ShOSM	π	κ	α	δ
2	mNHg1oC010p8w	ϖ	ω	α	δ
3	WslaCvfyF1Dxw	ξ	λ	α	ν
4	JpO8eLTVgwV1E	ρ	ν	β	γ
5	qctG6XnFNDTQc	ι	μ	α	σ
6	kotG8XnFNDTaW	χ	$\omicron$	β	ψ

Partition based index

Vediamo un esempio con la partizione, abbiamo un dominio, che è un budget che abbiamo. Partizioniamo i valori per cui tutti i valori in un range stanno in vari indici:



- $Map_{Balance}(100) = \mu$
- $Map_{Balance}(200) = \kappa$
- $Map_{Balance}(300) = \eta$
- $Map_{Balance}(400) = \theta$

Questa sicuramente è un indicizzazione con collisione. Vuol dire che partizioni il dominio in bucket, a ciascuno di questi dai un etichetta e quello che succede che nelle tuple il valore in chiaro è sostituito dall'etichetta. C'è una funzione ident che mappa la partizione. Così posso fare:

- Query di range
- Query di uguaglianza: posso vedere a che identificatore di partizione si mappa il valore che stai cercando. $MAP_{cond}(A_i = v) \rightarrow I_i = MAP$
- Le query di disuguaglianza:
 - Indice order preserving: vado a vedere dove si mappa il valore e poi cerco gli indici maggiori o minori di quelli
 - Indice non order preserving: cerco in tutte le partizioni in cui si può mappare tutte le partizioni minori di quello del valore
- Query attributo uguale attributo: dovrei guardare tutte le combinazioni indice-indice in cui potrei avere lo stesso valore

6.1.5 Query execution

Dobbiamo fare query che operano sugli indici dove sappiamo già che abbiamo il problema che se abbiamo collisione sull'indice il risultato è un super insieme di quello che stiamo cercando quindi abbiamo post-processing da fare. Quindi abbiamo una query da eseguire lato server ed una lato client sul risultato che abbiamo portato a casa.

Accounts			Accounts ₂ ^k				
Account	Customer	Balance	Counter	Etuple	I _A	I _C	I _B
Acc1	Alice	100	1	x4Z3ifX2ShOSM	π	α	μ
Acc2	Alice	200	2	mNHg1oC010p8w	ϖ	α	κ
Acc3	Bob	300	3	WslaCvfyF1Dxw	ξ	δ	θ
Acc4	Chris	200	4	JpO8eLTVgwV1E	ρ	α	κ
Acc5	Donna	400	5	qctG6XnFNDTQc	ς	β	κ
Acc6	Elvis	200	6	4QbqC3hxZHkIU	ι	β	κ

Original query on Accounts	Translation over Accounts ₂ ^k
Q := SELECT * FROM Accounts WHERE Balance=200	Q _s := SELECT Etuple FROM Accounts ₂ ^k WHERE I _B = κ Q _c := SELECT * FROM Decrypt(Q _s , Key) WHERE Balance=200

In questo esercizio andiamo ad interrogare gli indici della tabella blob, chiede di fare vedere com'è una tabella indicizzata, l'errore che vede è che si mettono le colonne separate degli indici o come colonne unite della tabella, invece no devono essere unite, perchè vado a querare gli indici per portarmi a casa la tupla corrispondente. Lato server diciamo di dare le tuple in cui $I_B = k$ quindi ci portiamo a casa delle tuple che non sono anche delle tuple che non vorremmo quindi a questo punto decriptiamo e facciamo la nostra query buttando via la tupla spuria.

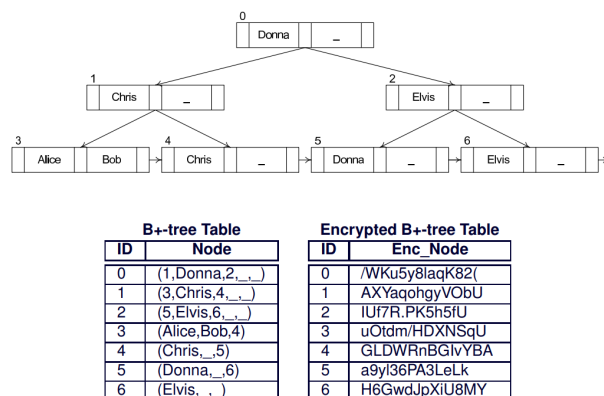
6.1.6 Hash based index

Se facciamo un hasing quello che vogliamo garantire è:

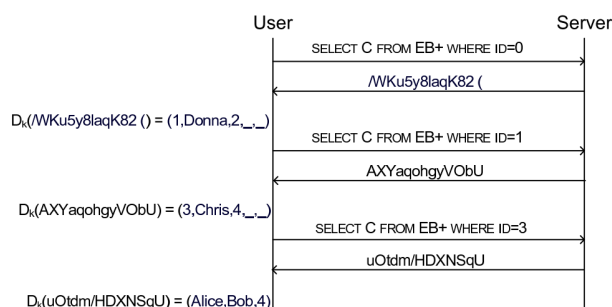
- Determinismo: non andare a fare una distribuzione, uno stesso valore è sempre mappato su un indice (due occorrenze di 200 sullo stesso indice)
- Protezione dal punto di vista dell'attacco di frequenza, non esporre frequenze: vogliamo collisione
- Strong mixing: la probabilità che due valori vicini siano mappati rispetto alla differenza dell hashing è uniforme, la distanza tra due valore in chiaro è indipendente dalla distanza dei valori di hashing.

Dal punto di vista di quello che vediamo nella tabella blob è la stessa cosa, vediamo dei valori indici e se abbiamo collisione tanti valori sono mappati sullo stesso indice. La traduzione delle query funziona allo stesso modo di prima.

Attenzione che fare l'indicizzazione lato server per avere indicizzazione delle query non ha senso, perchè la relazione tra gli indici non si riflette sui valori in chiaro, Quindi teoricamente una possibile struttura per l'indicizzazione va fatta da noi. Abbiamo il B+tree che funziona così, partiamo dalla radice ci portiamo a casa l'indice lo decriptiamo e capiamo da che figlio vogliamo andare e così via:



E attenzione che è chiaro che se costruisco una struttura di indicizzazione mia e la metto sul server da percorrerla significa che il percorso è avanti ed indietro perchè dobbiamo chiedere un nodo e poi l'altro:



6.1.7 Order preserving encryption

Se ho un indice order preserving o un encryption order preserving sto dando le stesse funzionalità, supportiamo le query di range sui dati criptati. Chiaro che esponiamo queste cose qui ai terzi. Se non abbiamo collisione non abbiamo tuple spurie.

6.2 Inference exposure

Dobbiamo parlare di cosa esponiamo sia in caso di encryption che di indici, c'è un possibile modo per modellare cosa stiamo esponendo. Se si fa indicizzazione diretta (o encryption diretta) se qualcuno ha accesso al disco ci frega.

Supponiamo di applicare un rapporto 1:1 o un rapporto con collisione n:1. Troviamo due scenari:

- $Freq + DB^k$: se qualcuno vede i nostri dati criptati e indicizzati (non il server) vede uno storage protetto per funzionalità e cosa sa? si parla di distribuzione dei valori per fare attacchi di frequenza. Può scoprire la funzione di indicizzazione e il contenuto (la tupla) perchè quando sai le frequenze potresti arrivarci
- $DB + DB^k$: Posso essere nel caso peggiore in cui l'attaccante vede i dati in chiaro sia i dati criptati. Potrebbe scoprire la funzione di indicizzazione e se scopre questa qualsiasi cosa inseriamo dopo lui la sa.

Knowledge

Account	Customer	Balance
Acc1	Alice	100
Acc2	Alice	200
Acc3	Bob	300
Acc4	Chris	200
Acc5	Donna	400
Acc6	Elvis	200

Accounts^k

Counter	Etuple	I _A	I _C	I _B
1	x4Z3tfX2ShOSM	π	α	μ
2	mNHg1oC010p8w	ϖ	α	κ
3	WslacvfyF1Dxw	ξ	β	η
4	JpO8eLTVgwV1E	ρ	γ	κ
5	qctG6XnFNDTQc	ς	δ	θ
6	4QbqC3hxZHkIU	ι	ε	κ

Inference

- $I_A = \text{Account}$
- $I_C = \text{Customer}$
- $I_B = \text{Balance}$
- $\kappa = 200$ (indexing inference)
- $\alpha = \text{Alice}$ (indexing inference)
- $\langle \text{Alice}, 200 \rangle$ is in the table (association inference)
- Alice is also associated with a value different from 200 ("100,300,400", all equiprobable)

Partiamo dalla conoscenza che i codici conto sono chiave, Alice ha due conti e ci sono 3 conti con 200, cosa possiamo scoprire? notiamo che nell'indice I_A abbiamo usato indicizzazione 1:1 e quindi sarà il conto, I_B è il customer e I_C il balance, sappiamo che κ è 200 perchè compare due volte e α è Alice perchè compare due volte, sappiamo che Alice, 200 è nella tabella perchè ha un conto che è di 200. Se vogliamo modellare quello che esponiamo con questo ragionamento stiamo esponendo le frequenze, Alice è esposta non perchè ha frequenza 2, ma perchè è l'unico con quella frequenza. Quelli che hanno lo stesso numero di frequenze si confondono tra di loro.

Vediamo la formula matematica, notiamo che tutti gli attributi chiave sono sempre protetti. Quotient table è la classe di equivalenza dell'indice, nella tabella inverse cardinality c'è dentro $1 / \text{la cardinalità della classe di equivalenza in cui sei}$, sostanzialmente dove vedo 1 ho i valori esposti quelli frazionari sono l'incertezza che ho. Se devo esprimere l'incertezza a livello di tupla è il prodotto dell'incertezza sui vari valori. Dal punto di vista della tabella andiamo a sommare sulle diverse tuple e poi dividiamo, facciamo la media dell'esposizione:

$$A.1 = \{\pi, \varpi, \xi, \rho, \varsigma, \iota\} = \{\text{Acc1}, \dots, \text{Acc6}\}$$

$$C.1 = \{\beta, \gamma, \delta, \varepsilon\} = \{\text{Bob}, \text{Chris}, \text{Donna}, \text{Elvis}\}$$

$$C.2 = \{\alpha\} = \{\text{Alice}\}$$

$$B.1 = \{\mu, \eta, \theta\} = \{100, 300, 400\}$$

$$B.3 = \{\kappa\} = \{200\}$$

INDEX_VALUES			QUOTIENT			INVERSE CARDINALITY		
I_A	I_C	I_B	qt_A	qt_C	qt_B	ic_A	ic_C	ic_B
π	α	μ	A.1	C.2	B.1	1/6	1	1/3
ϖ	α	κ	A.1	C.2	B.3	1/6	1	1
ξ	β	η	A.1	C.1	B.1	1/6	1/4	1/3
ρ	γ	κ	A.1	C.1	B.3	1/6	1/4	1
ς	δ	θ	A.1	C.1	B.1	1/6	1/4	1/3
ι	ε	κ	A.1	C.1	B.3	1/6	1/4	1

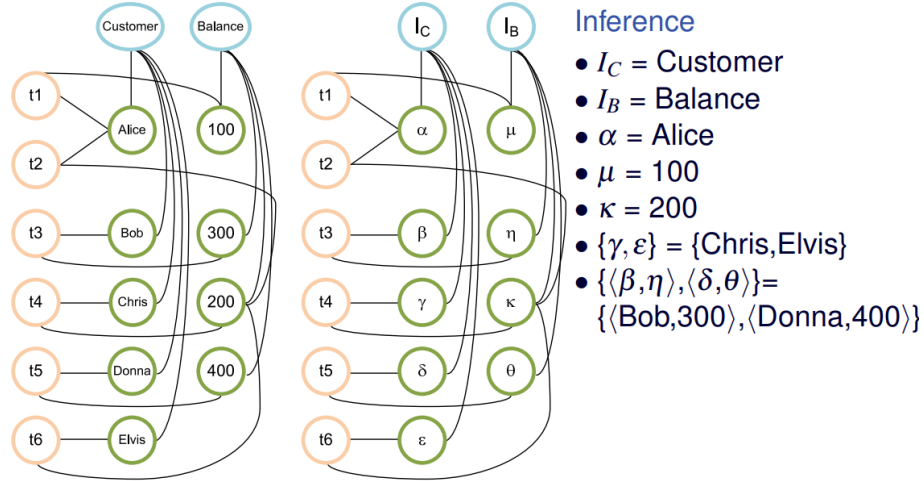
$$\mathcal{C} = \frac{1}{n} \sum_{i=1}^n \prod_{j=1}^k IC_{i,j} = 1/18$$

Vediamo l'esempio in cui l'attaccante sa i dati in chiaro e quelli criptati, potrebbe scoprire la funzione di indicizzazione, ad esempio non vorrei che tu sapessi che dietro κ c'è 200 e dietro β c'è Bob, perchè se in futuro dovessi mettere una tupla k, β sapresti che Bob ha un conto di 200:

Customer	Balance
Alice	100
Alice	200
Bob	300
Chris	200
Donna	400
Elvis	200

I_C	I_B
α	μ
α	κ
β	η
γ	κ
δ	θ
ε	κ

Per modellarlo bene viene bene usando un grafo che è la rappresentazione di quelle che sono le tuple, chiamato Row-Column-Value in cui abbiamo vertici di tipo riga, colonna e valore. Le tuple collegano i nodi di tipo colonna e riga ai nodi di tipo valore. Posso avere una misura di esposizione guardando gli isomorfismi del grafo, ovvero quanto un nodo si confonde, gli archi incidenti sui nodi valori e sui nodi indici sono uguali, quello che mi fa scoprire qualcosa è il numero di archi incidenti unici sul nodo:



Il grado di protezione non dipende solo da quanto si confondono i singoli nodi ma da quanto si confondono anche le associazioni. Quindi devo modellare quanto è esposto l'indice rispetto a delle associazioni che l'attaccante sa già. Quanto sono protetto è dato dagli automorphismi (isomorphismi del grafo). Di 9 indici che ho uno sta da solo, due sono confusi tra di loro, due sono confusi tra di loro, uno se ne sta da solo, due sono confusi tra di loro, uno se ne sta da solo. Quindi su 9 indici che abbiamo abbiamo 6 partizioni di nodi che si confondono tra di loro (se tutti fossero esposti avrei 9/9, se tutto confusi sarebbe 1/9):

$$\{(\alpha), (\beta, \delta), (\gamma, \epsilon), (\mu), (\eta, \theta), (k)\} \implies E = \frac{6}{9} = \frac{2}{3}$$

Se stiamo parlando di hashing, stiamo parlando di indicizzazione con collisione è più difficile per l'attaccante fare attacchi di associazione. Modellare indicizzazione con collisione si hanno altre modellazioni che si possono fare ma sono complicate e non ci chiederà mai queste cose.

6.3 Bloom Filter

Per indicizzare ragiona sulle funzioni hash ed è un modo per rappresentare la media degli insiemi. Abbiamo un insieme n grande di elementi e vogliamo mapparli in un vettore con un numero limitato di elementi, possiamo usare delle funzioni hash indipendenti. Teniamo un vettore di bit e se dobbiamo inserire un elemento calcoliamo per ciascuna delle tuple il valore della funzione di hash e metto a 1 il bit ritornata dalla funzione di hash:

Let $l = 10$ and $h = 3$

1	1			1		1		1	
1	2	3	4	5	6	7	8	9	10

- Insert **sun**: $H_1(\text{sun})=2$; $H_2(\text{sun})=5$; $H_3(\text{sun})=9$
- Insert **frog**: $H_1(\text{frog})=1$; $H_2(\text{frog})=5$; $H_3(\text{frog})=7$
- Search **dog**: $H_1(\text{dog})=2$; $H_2(\text{dog})=5$; $H_3(\text{dog})=10$
 $\Rightarrow$ No
- Search **car**: $H_1(\text{car})=1$; $H_2(\text{car})=5$; $H_3(\text{car})=9$
 $\Rightarrow$ Maybe Yes; **false positive!**

Per cercare un elemento vado a vedere quali sono i bit accesi se quell'elemento fosse li' dentro. Ho problemi quando ci sono delle collisioni, perche' potrebbero essere accesi gli stessi bit per parole diverse, ci sono falsi positivi dati dalle collisioni perche' ci sono diversi elementi per cui se guardo l'unione dei bit accesi da loro mi copro completamente i bit che accenderei io.

In pratica l'errore del falso positivo e' spesso accettato per l'efficienza in termini di spazio di questa struttura dati. Un ulteriore problema e' che gli elementi non possono essere rimossi perche' rischiamo di eliminare altri elementi.

6.4 Data Integrity

Noi abbiamo l'integrita' sullo storage e sulla computazione. Sullo storage significa dare certe garanzie che i dati non vengano modificati, devo assicurarmi che i dati non vengano modificati da utenti non autorizzati. La correttezza della computazione significa che il risultato query devono essere corrette e complete.

La soluzione dello storage e' firmare i dati, quello che potremmo fare e' chiamare tutte le tuple della tabella e firmare, mettere collegamenti tra i dati e firmare il totale (blockchain). Possiamo applicare a livello di tupla e quindi e' un costo che cresce con il numero di tuple.

6.5 Selective Encryption

Siamo nel contesto di voler avere un accesso fine granted, e' un accesso selettivo ai dati e ciascuno deve operare sui dati per i quali ha autorizzazione, c'e' il problema di cosa vuoi vedere e cosa puoi scrivere.

Vogliamo dare a diversi utenti diverse viste di accesso ai dati, queste cose ce le hai sulla base di qualcosa che fa il controllo dell'accesso (un reference monitor), in contesto classico centralizzato hai un'autenticazione e poi delle autorizzazioni ma ora l'autenticazione e

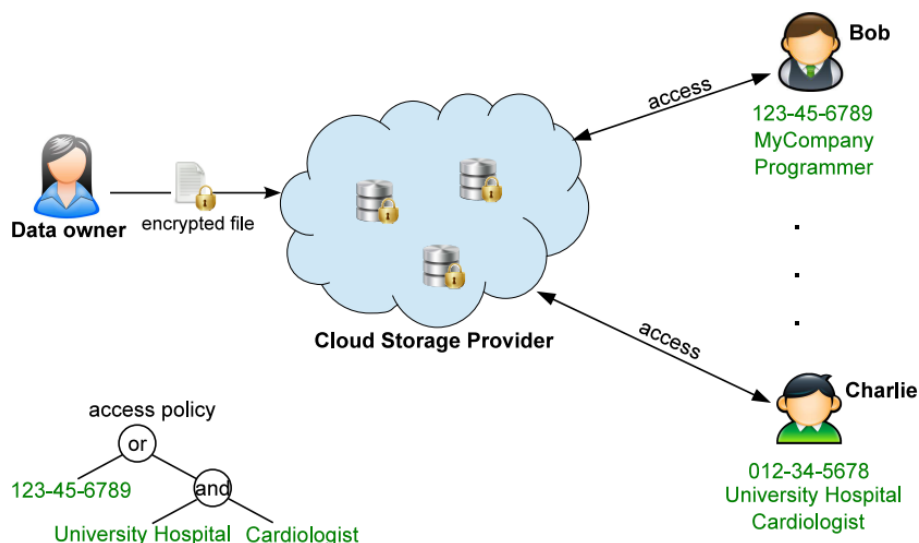
access control, come detto, sta perdendo questa forma rigida. Questo controllo dell'accesso se abbiamo i dati in cloud e' un problema, perche' vorremmo delegare l'access control, pero' vorrebbe dire delegare delle regolamentazioni. La direzione verso la quale si sta andando per cercare di far rimanere il data owner in controllo sulla regolamentazione dell'accesso e' combinare access control con l'encryption. Ci sono diversi modi di farlo.

6.5.1 Attribute base encryption

Abbiamo delle cose fuori nel cloud e vogliamo dare a diversi utenti viste diverse sui dati, lo facciamo usando crittografia a chiave pubblica (la chiave per criptare e decriptare e' la stessa, ho crittografia simmetrica, se ho due chiavi ho crittografia asimmetrica e pubblica) pero' le chiavi sono computate sulla base di attributi che mi permettono di implementare autorizzazione. Ci sono due versioni:

- KP-ABE
- CP-ABE

A seconda se la chiave la computo su un albero di attributi degli oggetti o soggetti. Dal nostro punto di vista non cambia, quello che succede e' che c'e' un'espressione della politica e computo la chiave per criptare i dati su questa politica (attributi). Questo dice che diversi oggetti saranno criptati con chiavi diverse e solo chi soddisfa quelle espressioni sara' in grado di decriptare i dati.



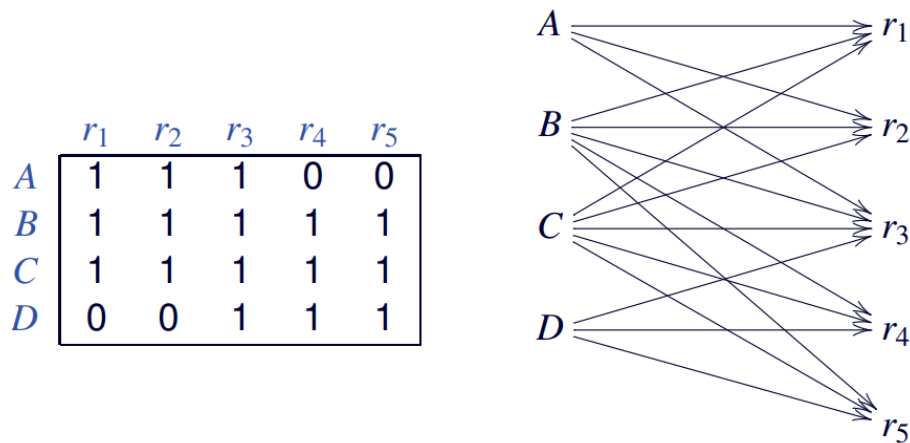
L'alternativa e' crittografia simmetrica in cui la chiave in cui cripto e che do' agli utenti e' sempre la stessa.

La differenza rispetto a prima e' che i dati che buttiamo fuori hanno tutti la stessa chiave e le chiavi ce le hanno gli utenti.

L'idea e' di implementare le autorizzazioni con dell'encryption selettiva, quindi i dati sono protetti perche' sono sempre criptati, e abbiamo autorizzazione usando la crittografia.

6.5.2 Politica di autorizzazione

Se vogliamo farla semplice la politica di autorizzazione e' una tupla soggetto, oggetto. Quindi dobbiamo dare accesso a ciascuno delle tuple che puo' vedere, le autorizzazioni possiamo darle come una matrice di accesso o come un grafo bipartito



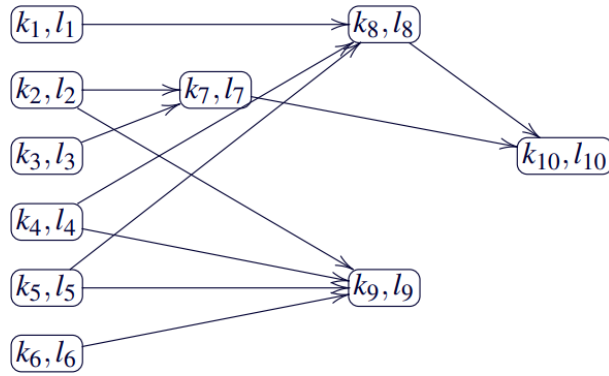
Quello che dobbiamo fare e' tradurre la politica di autorizzazione un un equivalente politica di encryption. Potremmo pensare di criptare ogni risorsa con una chiave e dare agli utenti le relative chiavi in base agli oggetti che possono vedere, oppure potremmo criptare per utente replicando le risorse per ogni utente significa che criptiamo le risorse piu' volte, brutta cosa.

Potremmo usare tecniche di encryption che permettono di fare derivazione di chiavi, possiamo avere una chiave in modo da derivare altre chiavi. Possiamo collegare le chiavi in gerarchia e la conoscenza di una chiave puo' permettere di derivarne altre. Abbiamo anche un ordinamento parziale delle chiavi, la conoscenza del link tra le chiavi puo' essere pubblicamente disponibile. Possiamo ordinarle a catena, grafo diretto aciclico o con un albero.

Stiamo parlando di crittografia simmetrica, ciascuna chiave avra' un'etichetta e poi il valore della chiave. IL collegamento che permette di derivare l'altra chiave e' fatto cosi': se abbiamo una chiave k_i possiamo derivare k_j se abbiamo un token che permette di derivare la chiave, il token lo specifica chi costruisce le chiavi, e' lo XOR tra k_j e l'hash di h_i con etichetta l_j .

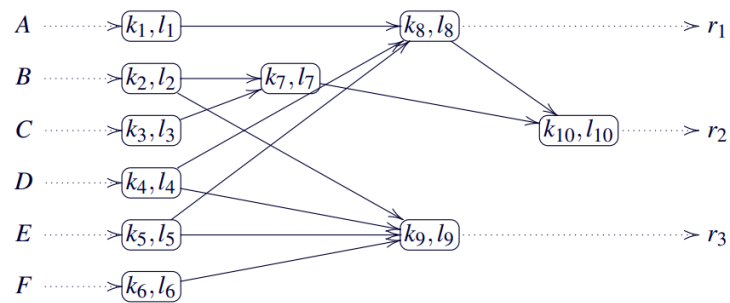
Le chiavi devono rimanere segrete ma i token no, possiamo pubblicarli con i dati sul server, quindi se abbiamo una chiave a questo punto potrei permetterti attraverso token che memorizzo sul server di farti avere tutte le chiavi che vuoi. Così avremmo per ogni oggetto una chiave sola.

I token ti permettono di mettere le chiavi in gerarchia:



Vogliamo avere una chiave per utente e ogni risorsa criptarla una volta, abbiamo una funzione che mi dice per ogni risorsa qual'è la sua chiave, quello che vogliamo ottenere è dalla politica di autorizzazione una politica di encryption. La politica di encryption ha un insieme di utenti, risorse, chiavi e etichette di chiavi, ha una funzione che associa a ciascuno utente la chiave e ad ogni risorsa la chiave, l'altro componente sono i token che stabiliscono il collegamento tra le chiavi.

Otteniamo un token graph e se assegniamo le chiavi agli utenti otteniamo la politica degli accessi:

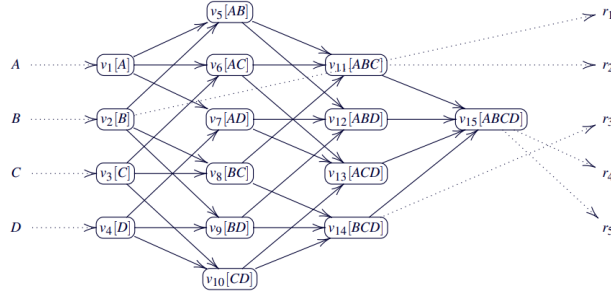


- user *A* can access $\{r_1, r_2\}$
- user *B* can access $\{r_2, r_3\}$
- user *C* can access $\{r_2\}$
- user *D* can access $\{r_1, r_2, r_3\}$
- user *E* can access $\{r_1, r_2, r_3\}$
- user *F* can access $\{r_3\}$

ϕ >
token ———>

Una politica di encryption equivalente alla politica di autorizzazione significa che un utente riesce a derivare le chiavi di tutte e sole le risorse che può vedere.

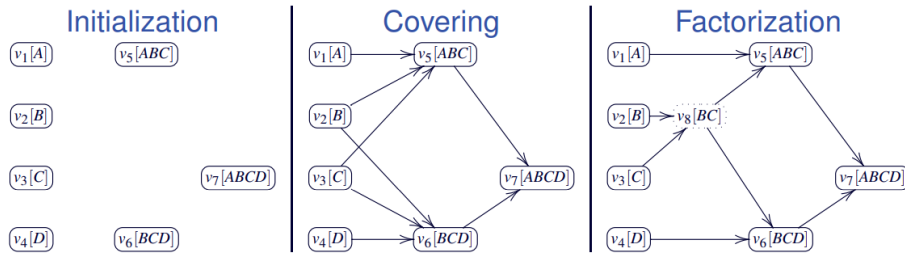
La cosa più facile che può venire in mente è una chiave per ogni utente e risorsa e un token per ogni arco del grafo. Ma non possiamo semplificare? sicuramente alcune risorse potranno essere lette da più utenti. Possiamo sfruttare che certi accessi sono fattorizzati quindi criptare le risorse con la stessa chiave se quelli che ci possono accedere sono gli stessi. Il peggio che ti va è specificare una chiave per ogni insieme di utenti:



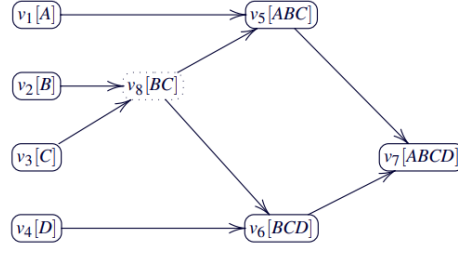
Osserviamo che tutti i gruppi di utenti che non corrispondono ad un ACL non hanno bisogno di una chiave. Dovremmo minimizzare le chiavi e i token. La prima osservazione e' che ci serve una chiave per ogni ACL, le altre potremmo farne a meno.

- Creare le chiavi che ti servono, per ogni utenti e un ACL non singleton (non corrisponde ad un utente) ti serve una chiave
- Dobbiamo legarci i token, possiamo mettere i token che sfrutta gli ACL. Per fare covering partiamo da nodi con cardinalita' maggiore e mettiamo i link da quelli dietro
- Se abbiamo un grafo in cui abbiamo un po' di nodi che vanno allo stesso nodo dell'altra parte possiamo metterci un nodo in mezzo in cui vanno tutti li' e poi da li' si va dall'altra parte tutti insieme. L'idea e' cercare di fattorizzare, se abbiamo un po' di chiavi che portano ad m chiavi metti in mezzo un token per fattorizzare

	r_1	r_2	r_3	r_4	r_5
A	0	1	0	1	1
B	1	1	1	1	1
C	0	1	1	1	1
D	0	0	1	1	1



A questo punto l'unica cosa che do agli utenti e' la loro chiave, mentre sul server ho i dati criptati e la funzione phi che dice ciascun utente quale chiave ha (che fa riferimento alle etichette), analogamente la phi facendo riferimento alle lable mi dice ciascuna risorsa che chiave ha. Sul server ci sono anche tutti i token che tanto possono essere pubblici:

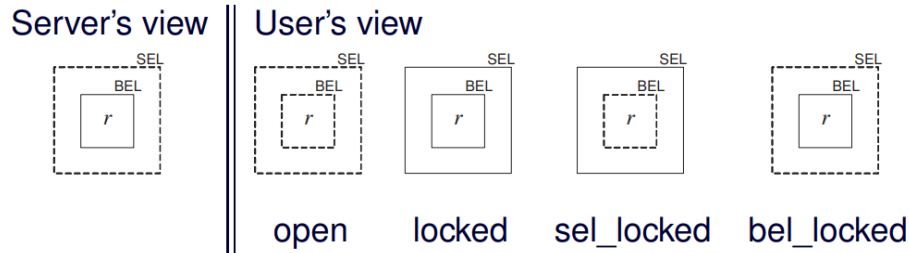


u	$\phi(u)$	r	$\phi(r)$	source	destination	token_value
A	$v_1.l$	r_1	$v_2.l$	$v_1.l$	$v_5.l$	$t_{1,5}$
B	$v_2.l$	r_2	$v_5.l$	$v_2.l$	$v_8.l$	$t_{2,8}$
C	$v_3.l$	r_3	$v_6.l$	$v_3.l$	$v_8.l$	$t_{3,8}$
D	$v_4.l$	r_4, r_5	$v_7.l$	$v_4.l$	$v_6.l$	$t_{4,6}$
				$v_5.l$	$v_7.l$	$t_{5,7}$
				$v_6.l$	$v_7.l$	$t_{6,7}$
				$v_8.l$	$v_5.l$	$t_{8,5}$
				$v_8.l$	$v_6.l$	$t_{8,6}$

6.6 Collusion Attack

Esiste ogni qual volta due parti mettendo insieme le conoscenze riescono ad andare oltre le conoscenze dei due. Ci dobbiamo preoccupare di collusioni tra utenti e con il server. Supponiamo che l'attaccante possa avere memoria.

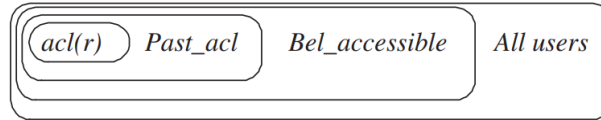
Abbiamo una risorsa che ha sostanzialmente due livelli di protezioni, quello dell'owner e quello dato dal server, la risorsa che è protetta da entrambi la chiamiamo locked, la risorsa che è scoperta su entrambi la chiamiamo aperta, non riesco a vedere la risorsa perché è lockata a livello sel ma a livello bel posso vederla o posso avere una risorsa lockata a livello bel ma aperta a livello sel (visione del server è sempre l'ultima).



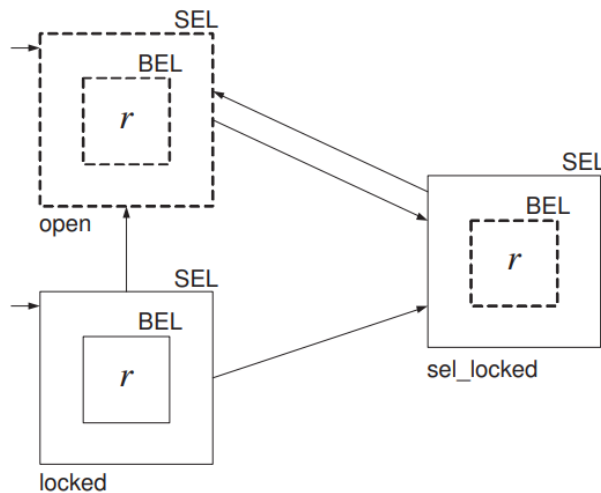
Alla vista sel_locked posso arrivarci per diverse ragioni, ad esempio se revochiamo un'autorizzazione l'utente la chiave BEL ce l'aveva.

6.6.1 Classification of users

Consideriamo una risorsa, possiamo essere uno autorizzato, uno non autorizzato ma autorizzato in passato, uno non autorizzato non autorizzato in passato ma la risorsa è scoperta oppure uno degli altri per cui la risorsa è comunque protetta. Il problema è se rispetto al passato mi pongo un problema, l'idea è che se la risorsa era criptata e non la ricripto allora non considero un problema che tu la veda di nuovo oppure potrei usare un'altra filosofia e ricriptare la risorsa.

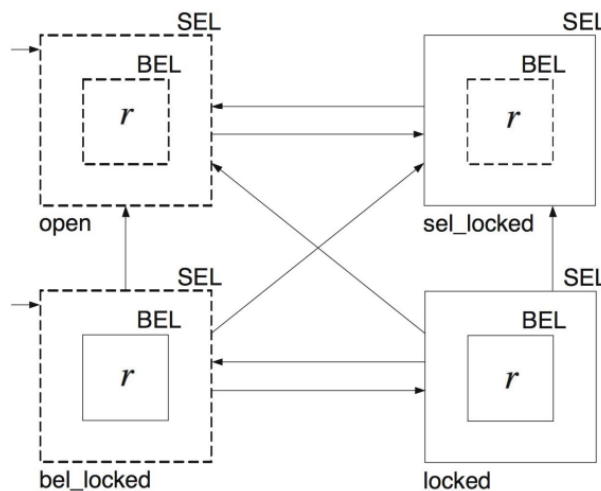


Vediamo il caso full.Sel, abbiamo due livelli di enctypion quelli autorizzati li passano entrambi quelli no nessuno questi sono i due punti di partenza della risorsa. Le frecce dicono quale può essere l'evoluzione della politica, potremmo potrare ad aprire le risorse oppure ad uno stato in cui la risorsa è lockata a livello bel ma esposta sel e da qui posso andare da una parte all'altra perchè il server può giocare con le sue chiavi:



Arriviamo ad essere sel_locker o perchè eravamo autorizzati poi revocata e la chiave sotto la conosciamo o perchè ci hanno dato una chiave che apre una risorsa a cui non potremmo accedere.

Vediamo il Delta_SEL, in partenza il server non protegge niente e gli utenti autorizzati saranno autorizzati a livello BEL dall'owner o no. Come transizioni di stato una risorsa può essere aperta:



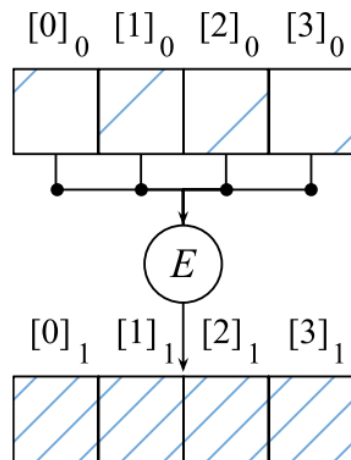
La collusione sugli utenti non è quasi mai un problema, il problema è la collusione con il server, perchè gli utenti sono aperti a livello sopra mentre il server può aprire la

risorsa. Quindi il delta sel è più efficiente però ha il problema dell'esposizione con i dati del passato, ad esempio se un utente al tempo 0 tira giù tutti i dati poi ad un tempo $t+x$ qualsiasi chiave gli venga esposta può decriptare dei dati a cui non dovrebbe poter accedere.

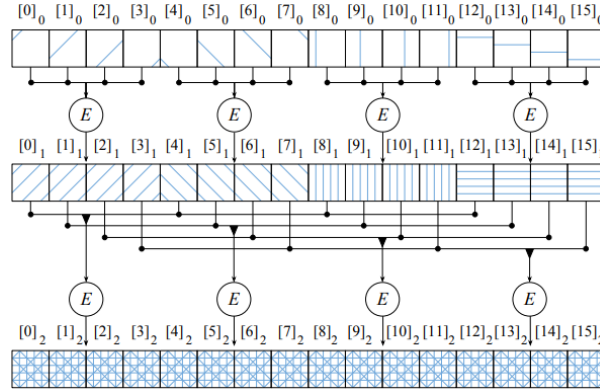
6.7 Mix & Slice

L'idea è di usare un encryption all or nothing, voglio criptare una risorsa per fare in modo che ciascun bit dell'input abbia effetto su ciascun bit dell'output quindi se ti manca un pezzettino non riesci a ricostruire niente della risorsa. Applico diversi round di encryption in modo che tutto si mischi con tutto. Si sfrutta questo perchè revocare significa semplicemente impedire la conoscenza di un pezzettino. Fare questo su risorse enormi e tagliarle a fette, fare un mixing tra le fette, le impilo, le frammento.

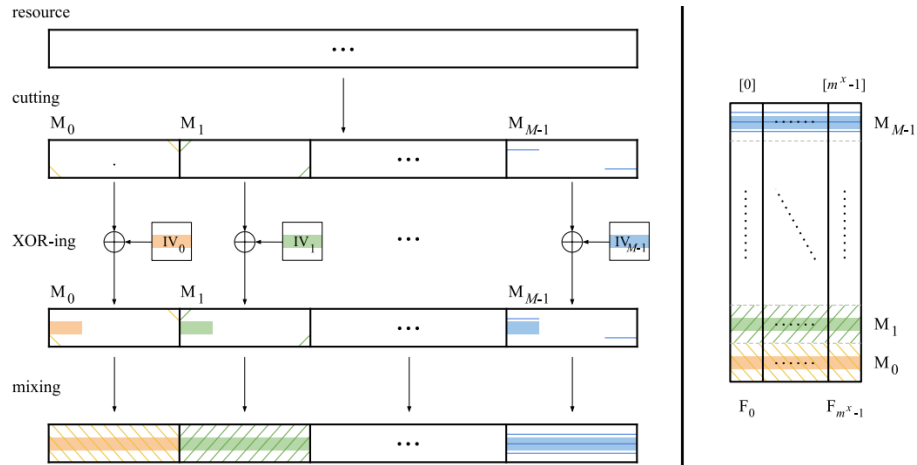
Vediamo un esempio, dividiamo la risorsa in macro-blocchi che criptiamo con la classica encryption, quello che abbiamo è che la funzione di criptazione mischia la roba che abbiamo nei blocchi (i bit nei blocchi), il problema è che anche se applichiamo la criptazione su una risorsa grande abbiamo mischiato ciascun macroblocco sono sostanzialmente pezzetti indipendenti, quello che sarebbe bello sarebbe avere questo all or nothing andando a criptare in modo che tutto dipenda da tutto per cui ciascun bit di input ha effetto su ciascun bit di output.



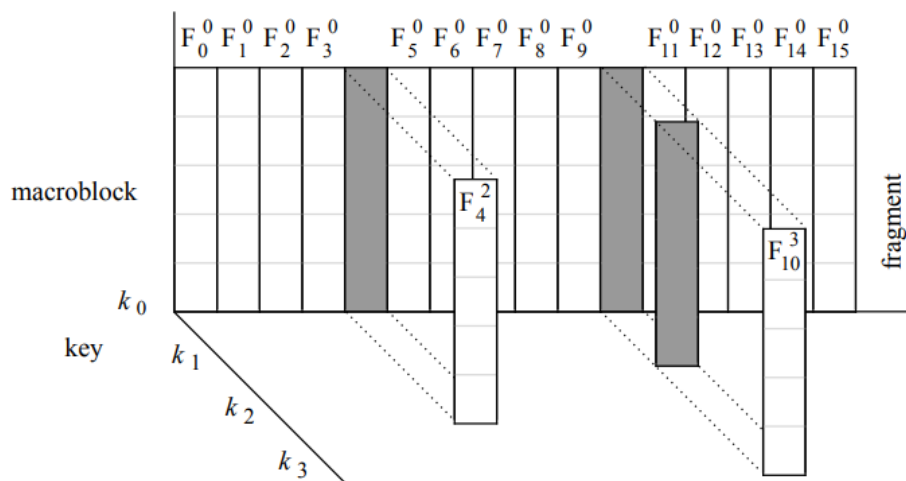
Per fare questa cosa al primo livello criptiamo a livello di blocco, adesso andiamo a fare la stessa cosa ma a pettine, prendiamo il primo micro-blocco di tutti e quattro i blocchi, il secondo microblocco, il terzo microblocco e il quarto microblocco di tutti e quattro e li mischiamo roba che viene dal primo blocco con roba da tutti gli altri i blocchi. Con due round abbiamo mischiato 16 blocchi, in generale con x round mischiamo 4^x blocchi:



Facendo così con risorse enormi ci metteremmo un sacco, con una risorsa enorme la tagliamo e facciamo diversi macro blocchi, ciascuno di questi è quello che opera nella prima fase precedente con un vettore di inizializzazione in XOR così sono tutti diversi in partenza, così sulla risorsa grande trovo tutti pezzi che soddisfano la proprietà di all or nothing, però sono sempre pezzi, quindi quello che faccio è considerare tutti i pezzi e impilarli e poi tagliamo in verticale la pila e questo è un frammento:



Significa che la risorsa la vedo come una lista di frammenti e ciascun frammento ha dentro le cose di tutti i macroblocchi, se ti tolgo un frammento qualsiasi ti mancherà un pezzettino di ciascun macroblocco che era stato mixato e quindi non riesci a ricostruire niente. Significa che per revocare la risorsa mi basta sovrascrivere un frammento. In realtà genero una nuova chiave che serve per risalire a quelle vecchie, quindi ricriptare significa criptare un frammento con una chiave nuova e dare quella chiave a chi può accedere. Rimango ancora esposto al fatto che la risorsa è statica e l'attaccante ha in casa la risorsa riesce a ricostruirla, siamo ancora esposti a collusione con l'attaccante e l'attaccante nel passato.



6.8 Write Authorizations

Se usi una crittografia simmetrica per proteggere la risorsa se sei in un contesto dove butti fuori i dati e tutti sono lettori questa cosa la puoi fare. Ma se la popolazione è di lettori e scrittori ti troverai a regolamentare delle autorizzazioni per leggere e scrivere con la stessa chiave, non va bene, dovremmo fare qualcosa di diverso. Chiaramente potremmo fare firmare i dati a chi li scrive.

Quello che si faceva qui è sfruttando l'hashing una derivazione di chiavi da presentare al server.

6.9 Fragmentation and Encryption

Il problema dell'encryption è garantire funzionalità quindi complica l'esecuzione delle query non tanto perchè sono criptati ma perchè il DBMS non è fidato e quindi non può aprire i dati per fare le query.

Potremmo chiederci cosa è confidenziale, perché l'encryption protegge quello che è confidenziale per noi, tutte le volte che quello che è sensibile non è tanto la visione sul valore ma sull'associazione di valori cryptare è un over do, quindi quello che si potrebbe fare è rompere quell'associazione. Se quello che è sensibile è l'associazione non criptare i valori ma proteggi l'associazione. Il problema è come proteggero l'associazione.

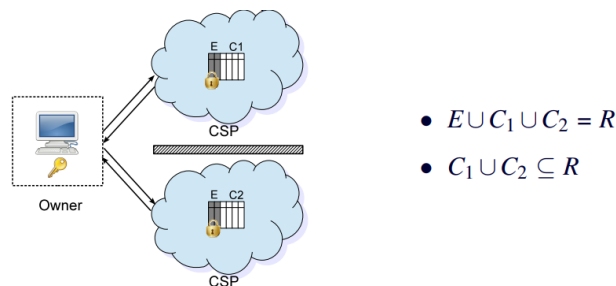
Ci sono dei valori che possono essere sensibili (come numero di telefono, indirizzi email) e in alcuni casi ho le associazioni sensibili.

$R = (\text{Name}, \text{DoB}, \text{Gender}, \text{Zip}, \text{Position}, \text{Salary}, \text{Email}, \text{Telephone})$

- $\{\text{Telephone}\}, \{\text{Email}\}$
 - attributes **Telephone** and **Email** are sensitive (cannot be stored in the clear)
- $\{\text{Name}, \text{Salary}\}, \{\text{Name}, \text{Position}\}, \{\text{Name}, \text{DoB}\}$
 - attributes **Salary**, **Position**, and **DoB** are private of an individual and cannot be stored in the clear in association with the name
- $\{\text{DoB}, \text{Gender}, \text{Zip}, \text{Salary}\}, \{\text{DoB}, \text{Gender}, \text{Zip}, \text{Position}\}$
 - attributes **DoB**, **Gender**, **Zip** can work as quasi-identifier
- $\{\text{Position}, \text{Salary}\}, \{\text{Salary}, \text{DoB}\}$
 - association rules between **Position** and **Salary** and between **Salary** and **DoB** need to be protected from an adversary

6.9.1 Coppia di server non comunicanti

É nata così, se ho un associazione di valori e l'associazione deve essere protetta uso due server mettendo un po' di dati di qua e di là, quindi sostanzialmente rompo l'associazione frammentando l'associazione. Mi può servire l'encryption anche in questo modo, perchè il problema è che ho due server e se ho associazioni di più di due elementi che non possono stare insieme devo comunque criptare qualcosa. Su ciascun server avrò dei dati criptati che a questo punto butto fuori, quindi dovrò avere che l'unione di tutto quello buttato fuori permette di ricostruire la tabella .



Quindi l'idea è costruire queste tabelle che separano le tuple, però ogni tabella deve avere un tuple id uguale per poter ricostruire la tabella originale, E è il numero di attributi criptato, e nessun constraint di confidenzialità può essere mantenuto in chiare in uno dei due server.

PATIENTS					
	SSN	Name	YoB	Job	Disease
t_1	123456789	Alice	1980	Clerk	Asthma
t_2	234567891	Bob	1980	Doctor	Asthma
t_3	345678912	Carol	1970	Nurse	Asthma
t_4	456789123	David	1970	Lawyer	Bronchitis
t_5	567891234	Eva	1970	Doctor	Bronchitis
t_6	678912345	Frank	1960	Doctor	Gastritis
t_7	789123456	Gary	1960	Teacher	Gastritis
t_8	891234567	Hilary	1960	Nurse	Diabetes

$c_0 = \{SSN\}$
 $c_1 = \{Name, Disease\}$
 $c_2 = \{Name, Job\}$
 $c_3 = \{Job, Disease\}$

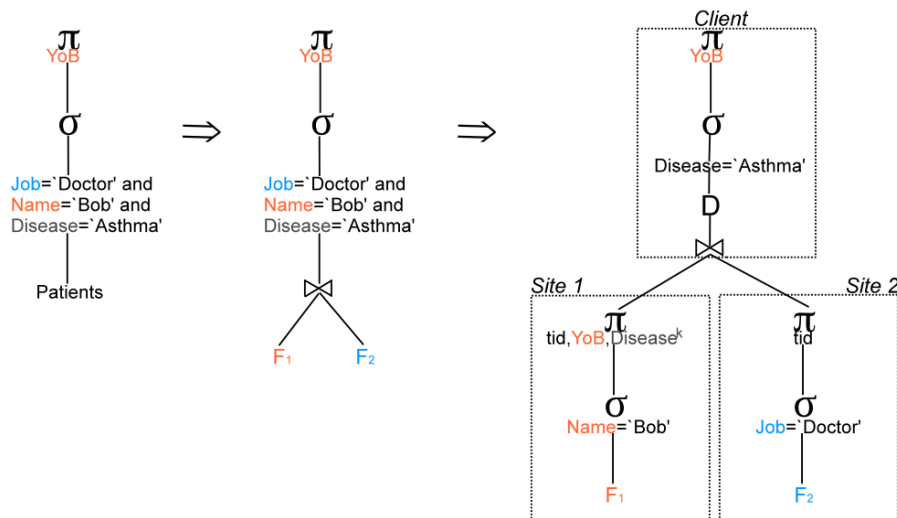
F_1				
tid	Name	YoB	SSN ^k	Disease ^k
1	Alice	1980	jdkis	hyaf4k
2	Bob	1980	u9hs9	j97;qx
3	Carol	1970	j9und	9jp'md
4	David	1970	p0vp8	p;nd92
5	Eva	1970	8nn[	0-mw-n
6	Frank	1960	j9jMK	wqp9[i
7	Gary	1960	87l'D	L0MB2G
8	Hilary	1960	8pm}n	@h8hwu

F_2			
tid	Job	SSN ^k	Disease ^k
1	Clerk	uwq8hd	jsd7ql
2	Doctor	j-0.dl;	0],nid
3	Nurse	8ojqdkf	j-0/?n
4	Lawyer	j0i12nd	5lkdpq
5	Doctor	mj[9;'s	j0982e
6	Doctor	aQ14l[	jnd%d
7	Teacher	8qsdQW	OP['
8	Nurse	0890UD	UP0D@

Come funzionano poi le query? devi ricostruire la tabella interrogando i due server. Il modo più banale di farlo è portare a casa tutte le tabelle fare il join e fare la query ma non conviene perchè quello che vogliamo è pushare nel server più operazioni possibili. Questa cosa si può fare in diversi modi, o fare le query in parallelo su entrambi i server oppure fare dei semi-join e poi filtro il risultato io.

Nell'esempio le prime due query sono equivalenti, mentre nell'ultima query voglio dal server arancio YoB il tid e il Disease (perchè mi serve per far la query dopo), dal server blu mi serve solo la tid per fare la query dopo:

$F_1: (tid, Name, YoB, SSN^k, Disease^k)$
 $F_2: (tid, Job, SSN^k, Disease^k)$



6.9.2 Identifying the optimal decomposition

Adesso il problema è come facciamo a fare la frammentazione, devo andare a vedere il workload delle query per capire dove mettere gli attributi o cosa lasciare in chiaro. Il workload può essere difficile da conoscere e da modellare, allora spesso quello che abbiamo rispetto al workload è la matrice di affinità ci dice quanto sia caro ad un attributo di stare con un altro attributo, significa quanto spesso quei due attributi sono acceduti e valutati insieme.

Se $M_{i,j}$ è quanto sia caro agli attributi di stare insieme è quanto costerebbe a noi separarli (paghiamo un overhead in esecuzione). L'identità $M_{i,i}$ è quanto spesso l'attributo è valutato nelle condizioni, se lo criptiamo dobbiamo decriptarlo ogni volta che valutiamo le condizioni.

Il nostro goal è fare una frammentazione che garantisce protezione andando a criptare il minimo possibile però cercando di impattare il minimo possibile l'esecuzione delle query (o lasciare in chiaro e insieme il più possibile gli attributi che ti servono per eseguire le query).

L'obiettivo è minimizzare questa sommatoria:

$$\sum_{i,j:i \in (R_1-E)j \in (R_2-E)} M_{i,j} + \sum_{i \in E} M_{i,i}$$

L'ultima sommatoria sono i componenti che sono criptati, vogliamo minimizzare gli elementi criptati, è la somma delle identità degli attributi criptati. La prima è la somma di affinità di attributi che sono stati separati da frammentazione.

Il problema di frammentazione è risolvere un problema di colorazione del grafo degli attributi. I constraint che abbiamo sono:

- Ciascun arco non tocca vertici dello stesso colore
- Minimizzare il peso dell'affinità tra gli attributi colpiti
- Per gli attributi soppressi devi guardare quanto costa la soppressione

La colorazione del grafo con due colori è un problema NP-hard, ci sono euristiche che risolvono il problema, vanno a vedere il min-cuts che è un taglio che minimizza gli archi colpiti e trovare il minimo insieme di sottoinsiemi che coprono l'insieme totale.

6.9.3 Multiple non-linkable fragments

Il problema di avere due server è anche di sicurezza, perchè ci sono due server che non si parlano e non si conoscono quindi non possiamo fidarci che i due server non si parlano e non colludono.

Allora togliamo il fatto che i due server non si parlano e facciamo una frammentazione che sia libera da collusione quindi il tuple id non deve esserci più.

Abbiamo una frammentazione che deve scegliere cosa mettere in chiaro in ogni frammento che possiamo avere, dove quello che vorrò è :

- Nessun frammento contiene completamente dei vincoli, vuol dire che nessun vincolo è esposto: $\forall F \in F, \forall c \in L : cF$

- I frammenti non devono avere attributi in comune: $\forall F_i, F_j \in F, i \neq j : F_i \cap F_j = \emptyset$

A livello fisico in un frammento ho in chiaro tutti gli attributi che per quella frammentazione sono in chiaro e avrò criptati tutti gli altri. Ci dice che ciascun frammento è mappato in un frammento fisico dove quello che ho è che gli attributi che appartengono a quel frammento sono lasciati in chiaro, il resto è un blob criptato che contiene tutti gli altri attributi che non sono qui dentro, l'unica cosa che dice è di salare la criptazione in modo che tuple uguale non sono riconosciute.

PATIENTS							
	SSN	Name	YoB	Job	Disease		
t_1	123456789	Alice	1980	Clerk	Asthma	$c_0 = \{\text{SSN}\}$	
t_2	234567891	Bob	1980	Doctor	Asthma	$c_1 = \{\text{Name, Disease}\}$	
t_3	345678912	Carol	1970	Nurse	Asthma	$c_2 = \{\text{Name, Job}\}$	
t_4	456789123	David	1970	Lawyer	Bronchitis	$c_3 = \{\text{Job, Disease}\}$	
t_5	567891234	Eva	1970	Doctor	Bronchitis		
t_6	678912345	Frank	1960	Doctor	Gastritis		
t_7	789123456	Gary	1960	Teacher	Gastritis		
t_8	891234567	Hilary	1960	Nurse	Diabetes		

F_1				F_2			F_3		
salt	enc	Name	YoB	salt	enc	Job	salt	enc	Disease
S_{11}	Bd6!l3	Alice	1980	S_{21}	8de6TO	Clerk	S_{31}	ew3)V!	Asthma
S_{12}	Oij3X.	Bob	1980	S_{22}	X'mIE3	Doctor	S_{32}	LkEd69	Asthma
S_{13}	9kEf6?	Carol	1970	S_{23}	wq.vy0	Nurse	S_{33}	w8vd66	Asthma
S_{14}	ker5/2	David	1970	S_{24}	nh=l3a	Lawyer	S_{34}	1"qPdd	Bronchitis
S_{15}	C:mE91	Eva	1970	S_{25}	hh%kj)	Doctor	S_{35}	(mn2eW	Bronchitis
S_{16}	4lDwqz	Frank	1960	S_{26}	,vf5eS	Doctor	S_{36}	wD}x1X	Gastritis
S_{17}	me3,op	Gary	1960	S_{27}	e4+YUp	Teacher	S_{37}	0opAuEI	Gastritis
S_{18}	zWf4g>	Hilary	1960	S_{28}	pgt6eC	Nurse	S_{38}	Sw@Fez	Diabetes

Query execution

Ciascuna query dovrebbe toccare un solo frammento, scegliamo il frammento in base alla selezione che screma più tuple, mi porto a casa il blob criptato che decrypto e da lì, dato che ho criptato tutta la tupla tranne il valore in chiaro, posso vedere tutto:

Original query on R	Translation over fragment F_3
$Q := \text{SELECT SSN, Name}$ FROM PATIENTS WHERE (Disease='Gastritis' OR Disease='Asthma') AND Job='Doctor'	$Q^3 := \text{SELECT salt, enc}$ FROM F_3 WHERE (Disease='Gastritis' OR Disease='Asthma') $Q' := \text{SELECT SSN, Name}$ FROM $\text{Decrypt}(Q^3, \text{Key})$ WHERE Job='Doctor'

Criteri di ottimizzazione

Sovra-frammentare è una brutta cosa, posso decidere come creare il numero di frammenti. Teoricamente la cosa migliore che potrei fare è usare il minor numero di frammenti e

sicuramente cercheremo di mettere insieme attributi che vogliono stare insieme sulla base del carico di lavoro. In generale tutti gli attributi che non è strettamente necessario non li cripti, al più ti crei un nuovo frammento.

Alla fine dei conti è un problema di minimizzazione dei frammenti, è come andare a colorare il grafo degli attributi minimizzando il numero di colori, questo è un problema noto. La frammentazione è minimale se unendo due qualsiasi dei frammenti violeremmo almeno un constraint.

Nell'esempio abbiamo 5 attributi da posizionare nei frammenti, proviamo ad ottenere una soluzione minimale, si è preso un attributo alla volta e si è visto in che frammento poteva entrare (in base ai constraint), se in nessuno se ne creava uno nuovo:

MEDICALDATA						Confidentiality constraints
SSN	Name	DoB	Zip	Illness	Physician	
123-45-6789	Nancy	65/12/07	94142	hypertension	M. White	$c_0 = \{\text{SSN}\}$
987-65-4321	Ned	73/01/05	94141	gastritis	D. Warren	$c_1 = \{\text{Name, DoB}\}$
963-85-2741	Nell	86/03/31	94139	flu	M. White	$c_2 = \{\text{Name, Zip}\}$
147-85-2369	Nick	90/07/19	94139	asthma	D. Warren	$c_3 = \{\text{Name, Illness}\}$
						$c_4 = \{\text{Name, Physician}\}$
						$c_5 = \{\text{DoB, Zip, Illness}\}$
						$c_6 = \{\text{DoB, Zip, Physician}\}$

Minimal fragmentation $\mathcal{F}$

- $F_1 = \{\text{Name}\}$
- $F_2 = \{\text{DoB, Zip}\}$
- $F_3 = \{\text{Illness, Physician}\}$

La matrice di affinità mi dice quanto due attributi vogliono stare insieme, se stanno insieme ti rispondo meglio alle query. Cominciamo mettendo dei pesi negativi per l'euristica in F_1 perchè non può andare con nessun altro, successivamente dato che siamo greedy prendiamo l'affinità più alta e mettiamo insieme i due elementi, scegliendo quell'elemento ci portiamo dietro anche le altre affinità dello stesso, successivamente si ricomputano le affinità mettendo un -1 per evitare di mettere F_3 e F_2 insieme:

Guarda slide 147.

Potrei fare qualcosa di meglio se sapessimo il workload, non solo potrei massimizzare rispetto all'affinità ma anche rispetto al carico di lavoro.

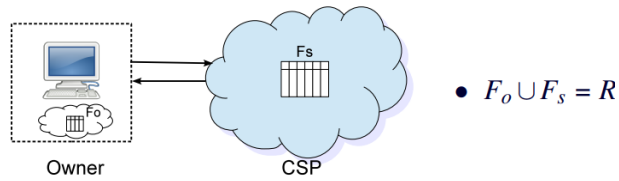
6.10 Fragmentation

L'ultimo flavour della frammentazione ci dice che potremmo buttare via l'encryption. Se siamo disposti a tenerci qualche dato in casa potremmo buttare fuori tutto in chiaro e quello che è privato e andrebbe criptato lo teniamo in caso, a questo punto facciamo a meno dell'encryption.

In realtà anche qui il gioco sarà buttare fuori il più possibile sia di computazione che di memoria. Ci troviamo con un frammento speciale e uno dell'owner, l'unione dei due devono ricostruire la tabella (tuple id).

Dati la tabella ed i constraint vogliamo determinare due frammenti dove:

- L'unione dei due frammenti ricostruisce la tabella originale



- Nel frammento visibile al server non ci deve essere nessun constraint completamente contenuto in un frammento
- L'intersezione dei due frammenti non può essere vuota, può essere rilassato per liberare spazio di storage, ma dal punto di vista della sicurezza ci dice che non è un vincolo di protezione

Dal punto di vista fisico avrò due frammenti con un tuple id che mi permette di fare una ricostruzione.

SSN lo devo tenere in casa e altri elementi in base ai vincoli:

PATIENTS					
	SSN	Name	YoB	Job	Disease
t_1	123456789	Alice	1980	Clerk	Asthma
t_2	234567891	Bob	1980	Doctor	Asthma
t_3	345678912	Carol	1970	Nurse	Asthma
t_4	456789123	David	1970	Lawyer	Bronchitis
t_5	567891234	Eva	1970	Doctor	Bronchitis
t_6	678912345	Frank	1960	Doctor	Gastritis
t_7	789123456	Gary	1960	Teacher	Gastritis
t_8	891234567	Hilary	1960	Nurse	Diabetes

F_o

tid	SSN	Job	Disease
1	123456789	Clerk	Asthma
2	234567891	Doctor	Asthma
3	345678912	Nurse	Asthma
4	456789123	Lawyer	Bronchitis
5	567891234	Doctor	Bronchitis
6	678912345	Doctor	Gastritis
7	789123456	Teacher	Gastritis
8	891234567	Nurse	Diabetes

F_s

tid	Name	YoB
1	Alice	1980
2	Bob	1980
3	Carol	1970
4	David	1970
5	Eva	1970
6	Frank	1960
7	Gary	1960
8	Hilary	1960

$C_0 = \{\text{SSN}\}$
 $C_1 = \{\text{Name, Disease}\}$
 $C_2 = \{\text{Name, Job}\}$
 $C_3 = \{\text{Job, Disease}\}$

6.10.1 Query evaluation

Come faccio a fare le query. Ho condizioni che coinvolgono attributi del client (la fa il client), del server (la fa il server) e insieme (la fa il server)

$F_o = \{\text{SSN}, \text{Job}, \text{Disease}\}, F_s = \{\text{Name}, \text{YoB}\}$

```
q = SELECT SSN, YoB
      FROM Patients
      WHERE (Disease="Bronchitis")
            AND (YoB="1970")
            AND (Name=Job)
```

The conditions in the WHERE clause are split as follows

- $C_o = \{\text{Disease} = \text{"Bronchitis"}\}$
- $C_s = \{\text{YoB} = \text{"1970"}\}$
- $C_{so} = \{\text{Name} = \text{Job}\}$

In questo caso va prima uno e poi l'altro, posso quindi avere due strategie:

- Server-client: Vado dal server, valuta la condizione restituisce e faccio il resto
- Client-Server: Valuto io, vado dal server, porto a casa risultato, faccio il resto

<pre>q = SELECT SSN, YoB FROM Patients WHERE (Disease = "Bronchitis") AND (YoB = "1970") AND (Name = Job)</pre>	<pre>C_o={Disease = "Bronchitis"} C_s={YoB = "1970"} C_so={Name = Job}</pre>
---	--

```
q_s = SELECT tid, Name, YoB
      FROM F_s
      WHERE YoB = "1970"
```

```
q_so = SELECT SSN, YoB
       FROM F_o JOIN r_s
           ON F_o.tid=r_s.tid
       WHERE (Disease = "Bronchitis") AND (Name = Job)
```

```

q = SELECT SSN, YoB
      FROM Patients
      WHERE (Disease = "Bronchitis")
            AND (YoB = "1970")
            AND (Name = Job)

```

```

Co={Disease = "Bronchitis"}
Cs={YoB = "1970"}
Cso={Name = Job}

```

```

qo = SELECT tid
      FROM Fo
      WHERE Disease = "Bronchitis"

```

```

qs = SELECT tid, Name, YoB
      FROM Fs JOIN ro ON Fs.tid=ro.tid
      WHERE YoB = "1970"

```

```

qso = SELECT SSN, YoB
      FROM Fo JOIN rs ON Fo.tid=rs.tid
      WHERE Name = Job

```

6.10.2 Minimal fragmentation

Il goal dovrebbe essere di uscire il più possibile l'overhead al server. Data una certa tabella soggetta ad un workload decidere cosa tenere in casa o fuori in modo che le query le faccia il server.

Vogliamo creare un frammento dell'owner e uno del server. Sappiamo che una frammentazione è minimale se:

- É corretta
- Vorrei minimizzarlo rispetto ad un qualche criterio che ora andiamo a definire

I criteri che puoi applicare sono:

- Storage: tenere in casa il minor numero di attributi possibili. Minimizzare la dimensione degli attributi, se un attributo pesa tanto butta fuori quello.
- Computation/traffic: minimizzare rispetto a quella che è l'esecuzione delle query. O rispetto a quanto devi pagare per la comunicazione con il server per portare a casa risultati.

Immaginiamo di avere due tabelle che ci dicono la size degli attributi e una del carico di lavoro con la frequenza di esecuzione delle query e che attributi usa:

PATIENT(SSN,Name,DoB,Race,Job,Illness,Treatment,HDate)

A	size(A)
SSN	9
Name	20
DoB	8
Race	5
Job	18
Illness	15
Treatment	40
HDate	8

q	freq(q)	Attr(q)	Cond(q)
q ₁	5	DoB, Illness	⟨DoB⟩, ⟨Illness⟩
q ₂	4	Race, Illness	⟨Race⟩, ⟨Illness⟩
q ₃	10	Job, Illness	⟨Job⟩, ⟨Illness⟩
q ₄	1	Illness, Treatment	⟨Illness⟩, ⟨Treatment⟩
q ₅	7	Illness	⟨Illness⟩
q ₆	7	DoB, HDate, Treatment	⟨DoB,HDate⟩, ⟨Treatment⟩
q ₇	1	SSN, Name	⟨SSN⟩, ⟨Name⟩

Per minimizzare gli attributi dobbiamo appunto minimizzare il numero di attributi che ci portiamo in casa, è sostanzialmente quanto di costa una frammentazione quanto il numero di elementi che tengo in casa.

Se invece vuoi minimizzare lo storage invece di prendere il numero di attributi che ho tenuto prendiamo la somma della loro dimensione.

Per minimizzare le query, conosciamo le query e per ciascuna la frequenza e gli attributi coinvolti, devi minimizzare la somma delle frequenze delle query che toccano attributi che ti sei tenuto in casa.

Diverso è il discorso se guardi le condizioni, perchè qui minimizzi la somma della frequenza delle condizioni che toccano attributi che ti sei tenuto in casa.